

Exercises

1. Write a complete Java program called `Stewie` that prints the following output:

```
////////////////////  
|| Victory is mine! ||  
\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\
```

2. Write a complete Java program called `Spikey` that prints the following output:

```
  \/  
  \\\/  
 \\\\/  
//\\/  
/\\/  
/  
/\\
```

3. Write a complete Java program called `wellFormed` that prints the following output:

```
A well-formed Java program has  
a main method with { and }  
braces.  
A System.out.println statement  
has ( and ) and usually a  
String that starts and ends  
with a " character.  
(But we type \" instead!)
```

4. Write a complete Java program called `Difference` that prints the following output:

```
What is the difference between  
a ' and a "? Or between a " and a \"?  
One is what we see when we're typing our program.
```

The other is what appears on the "console."

5. Write a complete Java program called `MuchBetter` that prints the following output:

```
A "quoted" String is
'much' better if you learn
the rules of "escape sequences."
Also, "" represents an empty String.
Don't forget: use \" instead of " !
' ' is not the same as "
```

6. Write a complete Java program called `Meta` whose output is the text that would be the source code of a Java program that prints “Hello, world!” as its output.
7. Write a complete Java program called `Mantra` that prints the following output. Use at least one static method besides `main`.

```
There's one thing every coder must understand:
The System.out.println command.
There's one thing every coder must understand:
The System.out.println command.
```

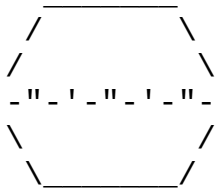
8. Write a complete Java program called `Stewie2` that prints the following output. Use at least one static method besides `main`.

```

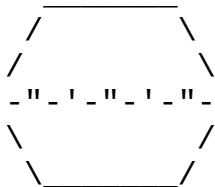
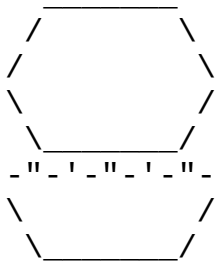
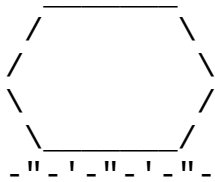
////////////////////////////////
|| Victory is mine! ||
////////////////////////////////
|| Victory is mine! ||
////////////////////////////////
|| Victory is mine! ||
////////////////////////////////
|| Victory is mine! ||
////////////////////////////////
|| Victory is mine! ||
////////////////////////////////

```

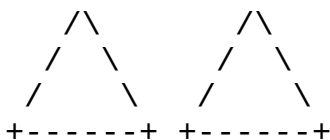
9. Write a program called Egg that displays the following output:



10. Modify the program from the previous exercise to become a new program `Egg2` that displays the following output. Use static methods as appropriate.



11. Write a Java program called `TwoRockets` that generates the following output. Use static methods to show structure and eliminate redundancy in your solution. Note that there are two rocket ships next to each other. What redundancy can you eliminate using static methods? What redundancy cannot be eliminated?



```

|       | |       |
+-----+ +-----+
|United| |United|
|States| |States|
+-----+ +-----+
|       | |       |
+-----+ +-----+
  /\      /\
 /\  \    /\  \
/\    \  /\    \

```

12. Write a program called `FightSong` that produces this output. Use at least two static methods to show structure and eliminate redundancy in your solution.

```

Go, team, go!
You can do it.
Go, team, go!
You can do it.
You're the best,
In the West.
Go, team, go!
You can do it.
Go, team, go!
You can do it.
You're the best,
in the West.
Go, team, go!
You can do it.
Go, team, go!
You can do it.

```

13. Write a Java program called `StarFigures` that generates the following output. Use static methods to show structure and eliminate redundancy in your solution.

```

*****
*****
 *  *
  *
*  *

```



```

* * * * *
* * * * *
  *  *
    *
  *  *
* * * * *
* * * * *
  *
  *
  *
* * * * *
* * * * *
  *  *
    *
  *  *

```

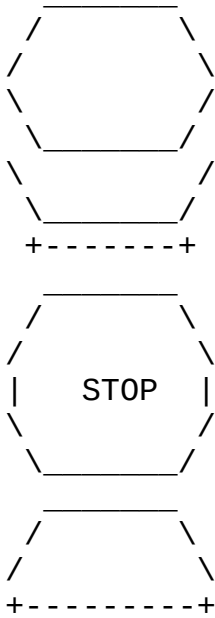
14. Write a Java program called `Lanterns` that generates the following output. Use static methods to show structure and eliminate redundancy in your solution.

```

  * * * * *
 * * * * *
* * * * *
  * * * * *
   * * * * *
  * * * * *
 * * * * *
*  |  |  |  |  *
* * * * *
  * * * * *
   * * * * *
  * * * * *
 * * * * *
* * * * *
  * * * * *
 *  |  |  |  |  *
*  |  |  |  |  *
  * * * * *
  * * * * *

```

15. Write a Java program called `EggStop` that generates the following output. Use static methods to show structure and eliminate redundancy in your solution.



16. Write a program called `Shining` that prints the following line of output 1000 times:

`All work and no play makes Jack a dull boy.`

You should not write a program that uses 1000 lines of source code; use methods to shorten the program. What is the shortest program you can write that will produce the 1000 lines of output, using only the material from this chapter?

Exercises

1. In physics, a common useful equation for finding the position s of a body in linear motion at a given time t , based on its initial position s_0 , initial velocity v_0 , and rate of acceleration a , is the following:

$$s = s_0 + v_0 t + \frac{1}{2} a t^2$$

Write code to declare variables for s_0 , v_0 , a , and t , and then write the code to compute s on the basis of these values.

2. Write a for loop that produces the following output:

```
1 4 9 16 25 36 49 64 81 100
```

For added challenge, try to modify your code so that it does not need to use the `*` multiplication operator. (It can be done! Hint: Look at the differences between adjacent numbers.)

3. The Fibonacci numbers are a sequence of integers in which the first two elements are 1, and each following element is the sum of the two preceding elements. The mathematical definition of each k th Fibonacci number is the following:

$$F(k) = \begin{cases} F(k-1) + F(k-2), & k > 2 \\ 1, & k \leq 2 \end{cases}$$

The first 12 Fibonacci numbers are

```
1 1 2 3 5 8 13 21 34 55 89 144
```

Write a for loop that computes and prints the first 12 Fibonacci numbers.

4. Write nested for loops to produce the following output:

```
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

5. Write nested for loops to produce the following output:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

6. Write nested for loops to produce the following output:

```
1  
22  
333  
4444  
55555  
666666  
7777777
```

7. Write nested for loops to produce the following output:

```
  1  
 2  
3  
4  
5
```

8. Write nested for loops to produce the following output:

```
  1  
 22  
333  
4444  
55555
```

9. Write nested for loops to produce the following output, with each line

40 characters wide:

```
-----  
_ ^ _ ^ _ ^ _ ^ _ ^ _ ^ _ ^ _ ^ _ ^ _ ^ _ ^  
1122334455667788990011223344556677889900  
-----
```

10. It's common to print a rotating, increasing list of single-digit numbers at the start of a program's output as a visual guide to number the columns of the output to follow. With this in mind, write nested for loops to produce the following output, with each line 60 characters wide:

```
      |      |      |      |      |      |  
123456789012345678901234567890123456789012345678901234567890
```

11. Modify your code from the previous exercise so that it could easily be modified to display a different range of numbers (instead of 1234567890) and a different number of repetitions of those numbers (instead of 60 total characters), with the vertical bars still matching up correctly. Use class constants instead of “magic numbers.” Here are some example outputs that could be generated by changing your constants:

```
      |      |      |      |      |      |      |      |      |  
1234012340123401234012340123401234012340123401234012340  
      |      |      |      |      |      |      |      |  
12345670123456701234567012345670123456701234567012345670
```

12. Write nested for loops that produce the following output:

```
000111222333444555666777888999  
000111222333444555666777888999  
000111222333444555666777888999
```

13. Modify the code so that it now produces the following output:

```
99999888887777766666555554444433333222221111100000  
99999888887777766666555554444433333222221111100000
```

```

99999888887777766666555554444433333222221111100000
99999888887777766666555554444433333222221111100000
99999888887777766666555554444433333222221111100000

```

14. Modify the code so that it now produces the following output:

```

999999999888888888777777666666555554444333221
999999999888888888777777666666555554444333221
999999999888888888777777666666555554444333221
999999999888888888777777666666555554444333221

```

15. Write a method called `printDesign` that produces the following output. Use nested for loops to capture the structure of the figure.

```

-----1-----
----333----
---5555---
--777777--
-99999999-

```

16. Write a Java program called `SlashFigure` that produces the following output. Use nested for loops to capture the structure of the figure. (See also Self-Check Problems 34 and 35.)

```

!!!!!!!!!!!!!!!!!!!!!!
\\!!!!!!!!!!!!!!!!!!!!//
\\\!!!!!!!!!!!!!!!!!!!!///
\\\\\\!!!!!!!!!!!!!!!!!!!!////
\\\\\\\\!!!!!!!!!!!!!!!!!!!!/////
\\\\\\\\\\!!!!!!!!!!!!!!!!!!!!/////
\\\\\\\\\\\\\\!!!!!!!!!!!!!!!!!!!!/////

```

17. Modify your `SlashFigure` program from the previous exercise to become a new program called `SlashFigure2` that uses a global constant for the figure's height. (You may want to make loop tables first.) The previous output used a constant height of 6. The following are the outputs for constant heights of 4 and 8:

Height 4

Height 8

```

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\\\!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\\\\\!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\\\\\\\!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\\\\\\\\\\\!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\\\\\\\\\\\\\\\!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\\\\\\\\\\\\\\\\\\\!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\\\\\\\\\\\\\\\\\\\\\\\!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\\\\\\\\\\\\\\\\\\\\\\\\\\\!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!//

```

18. Write a pseudocode algorithm that will produce the following figure as output:

```

+===+===+
|   |   |
|   |   |
|   |   |
+===+===+
|   |   |
|   |   |
|   |   |
+===+===+

```

19. Use your pseudocode from the previous exercise to write a Java program called `window` that produces the preceding figure as output. Use nested for loops to print the repeated parts of the figure. Once you get it to work, add a class constant so that the size of the figure can be changed simply by changing the constant's value.
20. Write a Java program called `StarFigure` that produces the following output. Use nested for loops to capture the structure of the figure.

```

//////////\\\\\\\\\\\\\\\\\\\\\\\\
//////////*****\\\\\\\\\\\\\\\\\\\\\\\\
//////////*****\\\\\\\\\\\\\\\\\\\\\\\\
//////////*****\\\\\\\\\\\\\\\\\\\\\\\\
//////////*****\\\\\\\\\\\\\\\\\\\\\\\\
*****

```

21. Modify your `StarFigure` program from the previous exercise to become a new program named `StarFigure2` that uses a global constant for the figure's height. (You may want to make loop tables first.) The previous

output used a constant height of 5. The following are the outputs for constant heights of 3 and 6:

Height 3

```

//////////\\\\\\\
//////*****\\
*****

```

Height 6

```

//////////\\\\\\\
//////////*****\\
//////////*****\\
//////////*****\\
//////////*****\\
//////////*****\\

```

22. Write a Java program called `DollarFigure` that produces the following output. Use nested for loops to capture the structure of the figure.

```

$$$$$$$*****$$$$$$$
**$$$$$$$*****$$$$$$$**
***$$$$$$$*****$$$$$$$***
*****$$$$$$$*****$$$$$$$*****
*****$$$$$$$*****$$$$$$$*****
*****$$$$$$$*****$$$$$$$*****
*****$$$$$$$*****$$$$$$$*****
*****$$$$$$$*****$$$$$$$*****

```

23. Modify your `DollarFigure` program from the previous exercise to become a new program called `DollarFigure2` that uses a global constant for the figure's height. (You may want to make loop tables first.) The previous output used a constant height of 7.

Exercises

1. Write a method called `printNumbers` that accepts a maximum number as an argument and prints each number from 1 up to that maximum, inclusive, boxed by square brackets. For example, consider the following calls:

```
printNumbers(15);  
printNumbers(5);
```

These calls should produce the following output:

```
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14]  
[1] [2] [3] [4] [5]
```

You may assume that the value passed to `printNumbers` is 1 or greater.

2. Write a method called `printPowersOf2` that accepts a maximum number as an argument and prints each power of 2 from 2^0 (1) up to that maximum power, inclusive. For example, consider the following calls:

```
printPowersOf2(3);  
printPowersOf2(10);
```

These calls should produce the following output:

```
1 2 4 8  
1 2 4 8 16 32 64 128 256 512 1024
```

You may assume that the value passed to `printPowersOf2` is 0 or greater. (The `Math` class may help you with this problem. If you use it, you may need to cast its results from `double` to `int` so that you don't see a `.0` after each number in your output. Also try to write this program without using the `Math` class.)

3. Write a method called `printPowersOfN` that accepts a base and an exponent as arguments and prints each power of the base from base^0 (1) up to that maximum power, inclusive. For example, consider the following calls:

```
printPowersOfN(4, 3);  
printPowersOfN(5, 6);  
printPowersOfN(-2, 8);
```

These calls should produce the following output:

```
1 4 16 64  
1 5 25 125 625 3125 15625  
1 -2 4 -8 16 -32 64 -128 256
```

You may assume that the exponent passed to `printPowersOfN` has a value of 0 or greater. (The `Math` class may help you with this problem. If you use it, you may need to cast its results from `double` to `int` so that you don't see a `.0` after each number in your output. Also try to write this program without using the `Math` class.)

4. Write a method called `printSquare` that accepts a minimum and maximum integer and prints a square of lines of increasing numbers. The first line should start with the minimum, and each line that follows should start with the next-higher number. The sequence of numbers on a line wraps back to the minimum after it hits the maximum. For example, the call

```
printSquare(3, 7);
```

should produce the following output:

```
34567  
45673  
56734  
67345  
73456
```

If the maximum passed is less than the minimum, the method produces no output.

5. Write a method called `printGrid` that accepts two integers representing a number of rows and columns and prints a grid of integers from 1 to (rows * columns) in column major order. For example, the call

```
printGrid(4, 6);
```

should produce the following output:

```
1 5 9 13 17 21
2 6 10 14 18 22
3 7 11 15 19 23
4 8 12 16 20 24
```

6. Write a method called `largerAbsVal` that takes two integers as parameters and returns the larger of the two absolute values. A call of `largerAbsVal(11, 2)` would return 11, and a call of `largerAbsVal(4, -5)` would return 5.
7. Write a variation of the `largestAbsVal` method from the last exercise that takes three integers as parameters and returns the largest of their three absolute values. For example, a call of `largestAbsVal(7, -2, -11)` would return 11, and a call of `largestAbsVal(24, 5, 2)` would return 5.
8. Write a method called `quadratic` that solves quadratic equations and prints their roots. Recall that a quadratic equation is a polynomial equation in terms of a variable x of the form $ax^2 + bx + c = 0$. The formula for solving a quadratic equation is

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Here are some example equations and their roots:

$$x^2 - 7x + 12: x = 4, x = 3 \quad x^2 - 3x + 2: x = -2, x = -1$$

Your method should accept the coefficients a , b , and c as parameters

and should print the roots of the equation. You may assume that the equation has two real roots, though mathematically this is not always the case.

9. Write a method called `lastDigit` that returns the last digit of an integer. For example, `lastDigit(3572)` should return 2. It should work for negative numbers as well. For example, `lastDigit(-947)` should return 7.
10. Write a method called `area` that accepts as a parameter the radius of a circle and that returns the area of the circle. For example, the call `area(2.0)` should return 12.566370614359172. Recall that area can be computed as pi (p) times the radius squared and that Java has a constant called `Math.PI`.
11. Write a method called `distance` that accepts four integer coordinates x_1 , y_1 , x_2 , and y_2 as parameters and computes the distance between points (x_1, y_1) and (x_2, y_2) on the Cartesian plane. The equation for the distance is

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

For example, the call of `distance(1, 0, 4, 4)` would return 5.0 and the call of `distance(10, 2, 3, 15)` would return 14.7648230602334.

12. Write a method called `scientific` that accepts a real number base and an exponent as parameters and computes the base times 10 to the exponent, as seen in scientific notation. For example, the call of `scientific(6.23, 5)` would return 623000.0 and the call of `scientific(1.9, -2)` would return 0.019.
13. Write a method called `pay` that accepts two parameters: a real number for a TA's salary, and an integer for the number of hours the TA worked this week. The method should return how much money to pay the TA. For example, the call `pay(5.50, 6)` should return 33.0. The TA should receive "overtime" pay of $1\frac{1}{2}$ times the normal salary for any hours above 8. For example, the call `pay(4.00, 11)` should return $(4.00 * 8) + 1$

(6.00 * 3) or 50.0.

14. Write a method called `cylinderSurfaceArea` that accepts a radius and height as parameters and returns the surface area of a cylinder with those dimensions. For example, the call `cylinderSurfaceArea(3.0, 4.5)` should return 141.3716694115407. The formula for the surface area of a cylinder with radius r and height h is the following:

$$\text{surface area} = 2\pi r^2 + 2\pi rh$$

15. Write a method called `sphereVolume` that accepts a radius as a parameter and returns the volume of a sphere with that radius. For example, the call `sphereVolume(2.0)` should return 33.510321638291124. The formula for the volume of a sphere with radius r is the following:

$$\text{volume} = \frac{4}{3}\pi r^3$$

16. Write a method called `triangleArea` that accepts the three side lengths of a triangle as parameters and returns the area of a triangle with those side lengths. For example, the call `triangleArea(8, 5.2, 7.1)` should return 18.151176098258745. To compute the area, use Heron's formula, which states that the area of a triangle whose three sides have lengths a , b , and c , is the following. The formula is based on the computed value s , a length equal to half the perimeter of the triangle:

$$\text{area} = s(s-a)(s-b)(s-c) \quad s = \frac{a+b+c}{2}$$

17. Write a method called `padString` that accepts two parameters: a string and an integer representing a length. The method should pad the parameter string with spaces until its length is the given length. For example, `padString("hello", 8)` should return "hello ". (This sort of method is useful when trying to print output that lines up horizontally.) If the string's length is already at least as long as the length parameter, your method should return the original string. For example, `padString("congratulations", 10)` should return "congratulations".

18. Write a method called `vertical` that accepts a string as its parameter and

prints each letter of the string on separate lines. For example, a call of `vertical("hey now")` should produce the following output:

```
h
e
y
n
o
w
```

19. Write a method called `printReverse` that accepts a string as its parameter and prints the characters in opposite order. For example, a call of `printReverse("hello there!")` should print `"!ereht olleh"`. If the empty string is passed, the method should produce no output.
20. Write a method called `inputBirthday` that accepts a `Scanner` for the console as a parameter and prompts the user to enter a month, day, and year of birth, then prints the birthdate in a suitable format. Here is an example dialogue with the user:

```
On what day of the month were you born? 8
What is the name of the month in which you were born? May
During what year were you born? 1981
You were born on May 8, 1981. You're mighty old!
```

21. Write a method called `processName` that accepts a `Scanner` for the console as a parameter and prompts the user to enter a full name, then prints the name in reverse order (i.e., last name, first name). Here is an example dialogue with the user:

```
Please enter your full name: Sammy Jankis
Your name in reverse order is Jankis, Sammy
```

22. Write a program that outputs “The Name Game,” where the user inputs a first and last name and a song in the following format is printed about their first, then last, name. Use a method to avoid redundancy.

```
What is your name? Fifty Cent
```

Fifty Fifty, bo-Bifty
Banana-fana fo-Fifty
Fee-fi-mo-Mifty
FIFTY!
Cent, Cent, bo-Bent
Banana-fana fo-Fent
Fee-fi-mo-Ment
CENT!

Exercises

1. Write a program that uses the DrawingPanel to draw [Figure 3G.21](#).

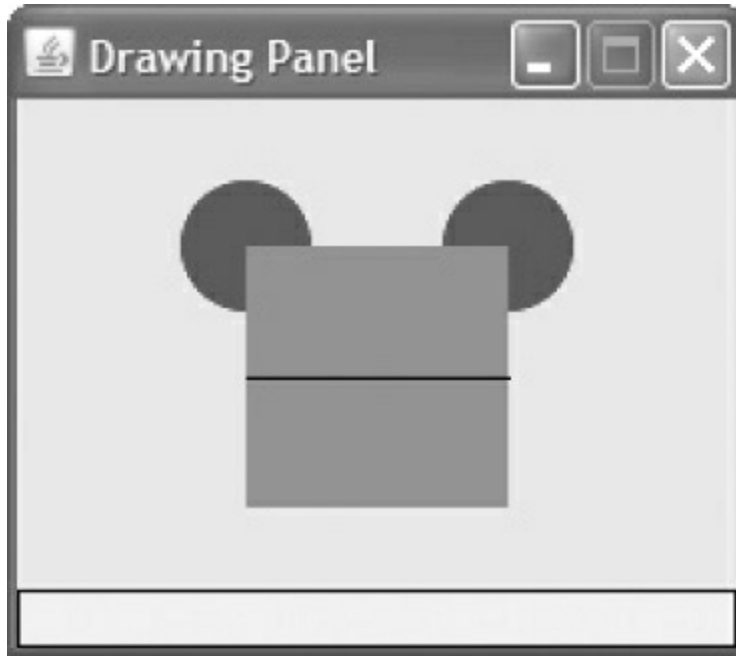


Figure 3G.21 Expected graphical output of Exercise 3G.1

The window is 220 pixels wide and 150 pixels tall. The background is yellow. There are two blue ovals of size 40 3 40 pixels. They are 80 pixels apart, and the left oval's top-left corner is located at position (50, 25). There is a red square whose top two corners exactly intersect the centers of the two ovals. Lastly, there is a black horizontal line through the center of the square.

2. Modify your program from the previous exercise to draw the figure by a

method called `drawFigure`. The method should accept three parameters: the `Graphics g` of the `DrawingPanel` on which to draw, and a pair of (x, y) coordinates specifying the location of the top-left corner of the figure. Use the following heading for your method:

```
public static void drawFigure(Graphics g, int x, int y)
```

Set your `DrawingPanel`'s size to 450 3 150 pixels, and use your `drawFigure` method to place two figures on it, as shown in [Figure 3G.22](#). One figure should be at position $(50, 25)$ and the other should be at position $(250, 45)$.

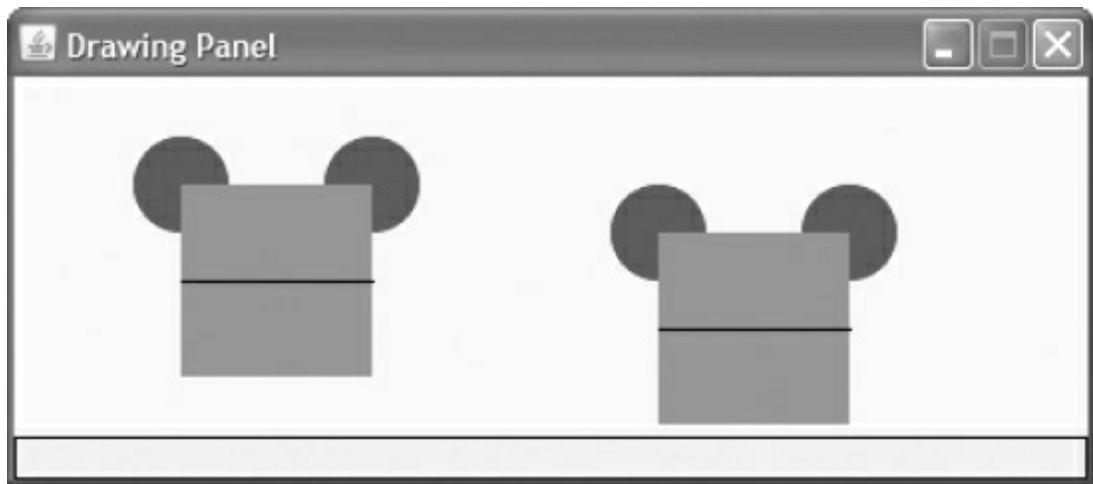


Figure 3G.22 Expected graphical output of Exercise 3G.2

3. Suppose you have the following existing program called `Face` that uses the `DrawingPanel` to draw the face figure shown in [Figure 3G.23](#). Modify the program to draw the modified output shown in [Figure 3G.24](#). Do so by writing a parameterized method that draws a face at different positions. The window size should be changed to 320 3 180 pixels, and the two faces' top-left corners are at $(10, 30)$ and $(150, 50)$.

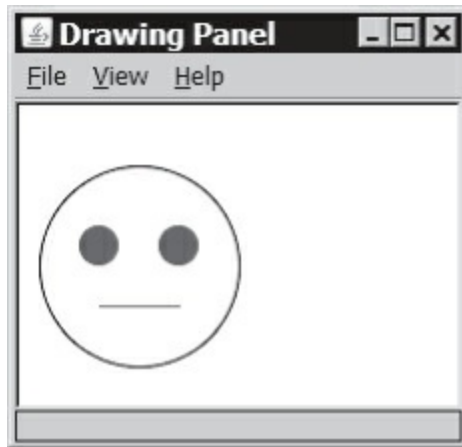


Figure 3G.23 Initial graphical output of Exercise 3G.3

```
1 public class Face {
2     public static void main(String[] args) {
3         DrawingPanel panel = new DrawingPanel(220, 150);
4         Graphics g = panel.getGraphics();
5
6         g.setColor(Color.BLACK);
7         g.drawOval(10, 30, 100, 100);    // face outline
8
9         g.setColor(Color.BLUE);
10        g.fillOval(30, 60, 20, 20);      // eyes
11        g.fillOval(70, 60, 20, 20);
12
13        g.setColor(Color.RED);           // mouth
14        g.drawLine(40, 100, 80, 100);
15    }
16 }
```

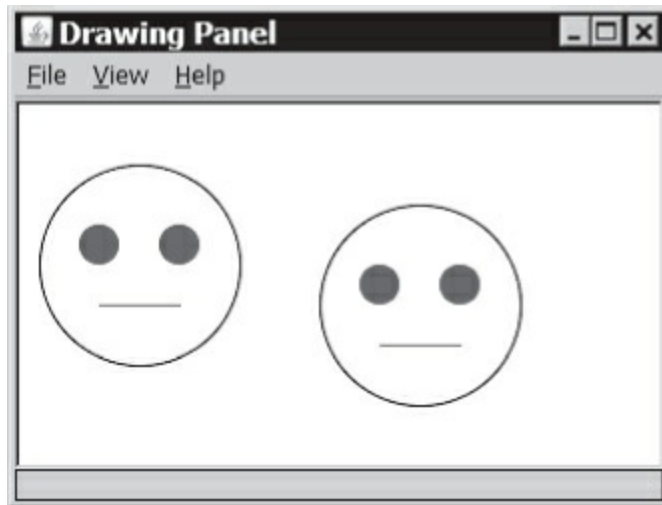


Figure 3G.24 Expected graphical output of Exercise 3G.3

4. Modify your previous Face program to draw the new output shown in [Figure 3G.25](#). The window size should be changed to 520 3 180 pixels, and the faces' top-left corners are at (10, 30), (110, 30), (210, 30), (310, 30), and (410, 30).



Figure 3G.25 Expected graphical output of Exercise 3G.4

5. Write a program called ShowDesign that uses the DrawingPanel to draw [Figure 3G.26](#).

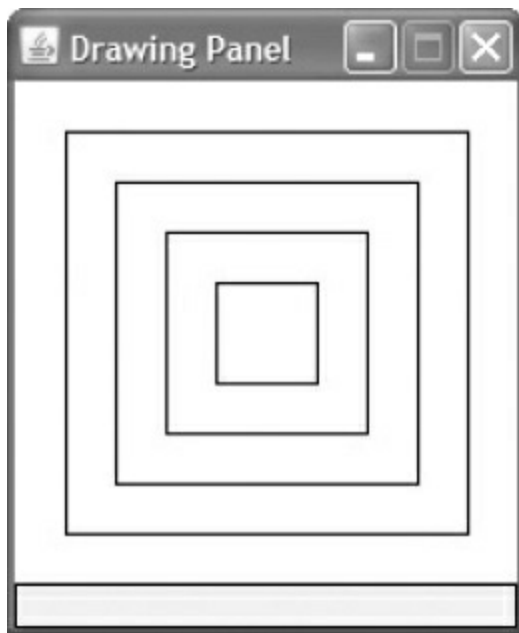


Figure 3G.26 Expected graphical output of Exercise 3G.5

The window is 200 pixels wide and 200 pixels tall. The background is white and the foreground is black. There are 20 pixels between each of the four rectangles, and the rectangles are concentric (their centers are at the same point). Use a loop to draw the repeated rectangles.

6. Modify your ShowDesign program from the previous exercise so that it has a method that accepts parameters for the window width and height and displays the rectangles at the appropriate sizes. For example, if your method was called with values of 300 and 100, the window would look like [Figure 3G.27](#).

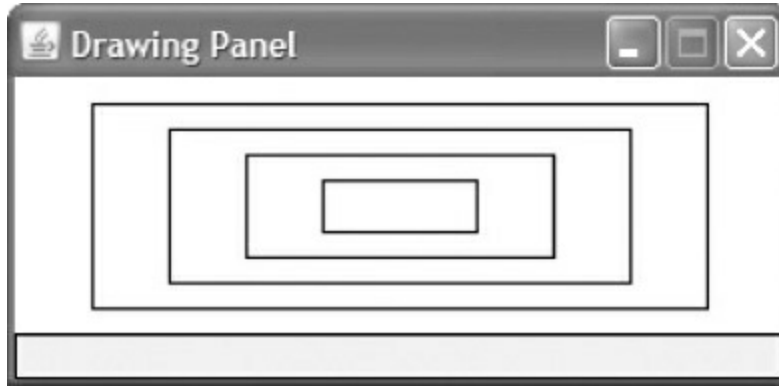


Figure 3G.27 Expected graphical output of Exercise 3G.6

7. Write a program called Squares that uses the DrawingPanel to draw the shape shown in [Figure 3G.28](#).

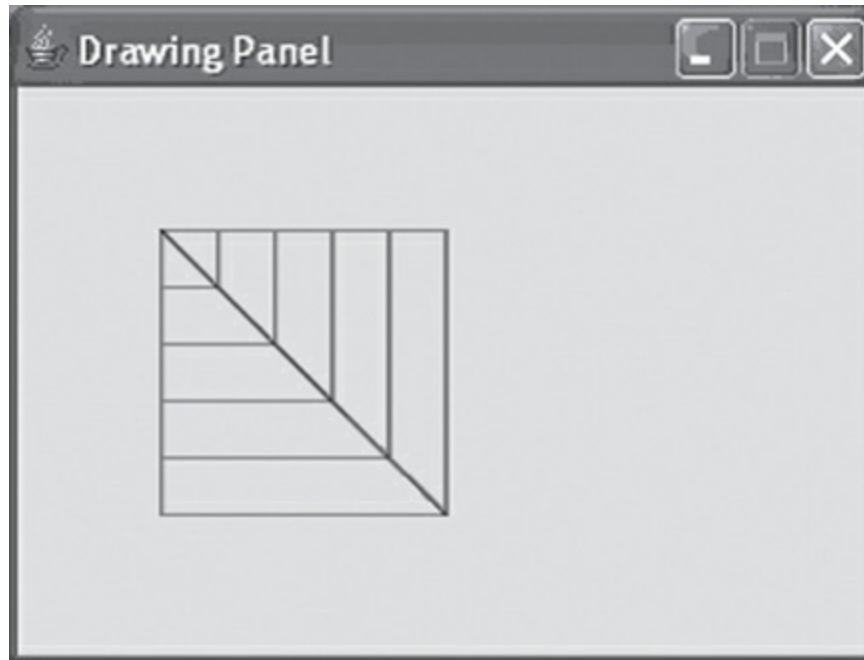


Figure 3G.28 Expected graphical output of Exercise 3G.7

The `DrawingPanel` is 300 pixels wide by 200 pixels high. Its background is cyan. The horizontal and vertical lines are drawn in red and the diagonal line is drawn in black. The upper-left corner of the diagonal line is at (50, 50). Successive horizontal and vertical lines are spaced 20 pixels apart.

8. Modify your code from the previous exercise to produce the pattern shown in [Figure 3G.29](#).

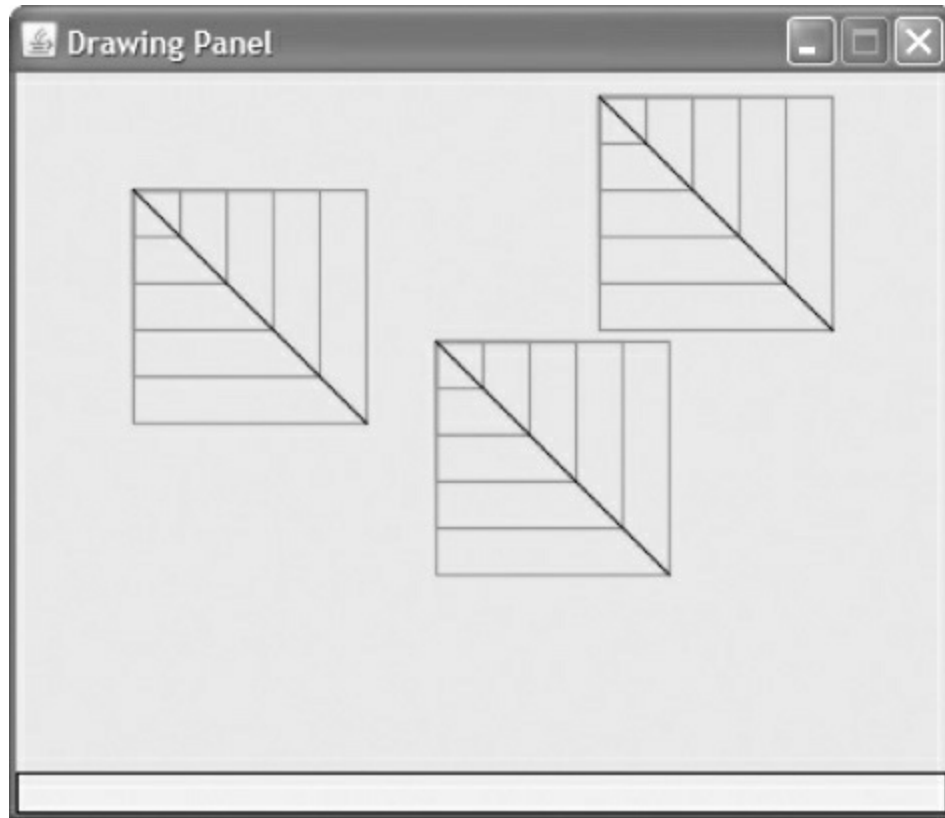


Figure 3G.29 Expected graphical output of Exercise 3G.8

The `DrawingPanel` is now 400 x 300 pixels in size. The first figure is at the same position, (50, 50). The other figures are at positions (250, 10) and (180, 115), respectively. Use one or more parameterized static methods to reduce the redundancy of your solution.

9. Modify your code from the previous exercise to produce the pattern shown in [Figure 3G.30](#).

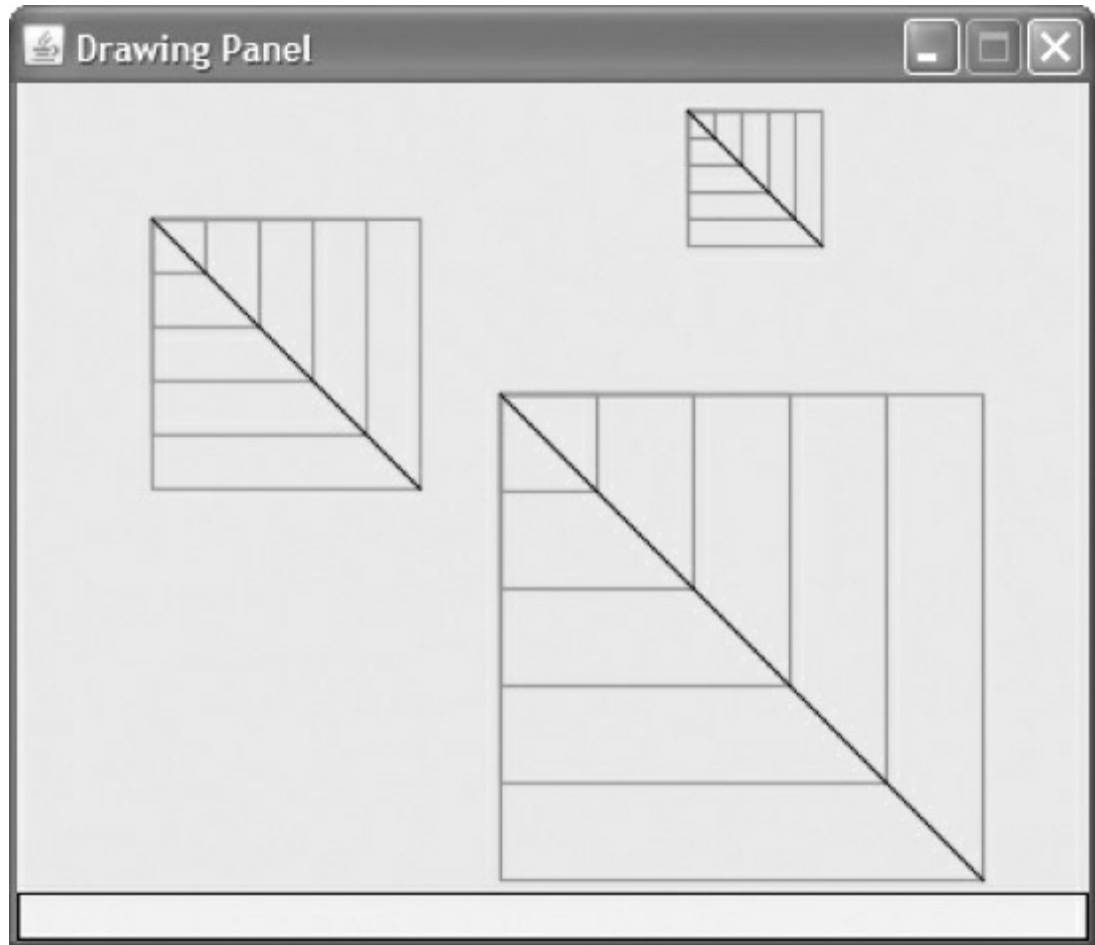


Figure 3G.30 Expected graphical output of Exercise 3G.9

The `DrawingPanel` is the same except that now each figure has a different size. The left figure has its original size of 100, the top-right figure has a size of 50, and the bottom-right figure has a size of 180. Use parameterized static methods to reduce the redundancy of your solution.

10. Write a program called `Stairs` that uses the `DrawingPanel` to draw the figure shown in [Figure 3G.31](#). The first stair's top-left corner is at position (5, 5). The first stair is 10 3 10 pixels in size. Each stair is 10

pixels wider than the one above it. Make a table with the (x, y) coordinates and (*width* 3 *height*) sizes of the first five stairs. Note which values change and which ones stay the same.

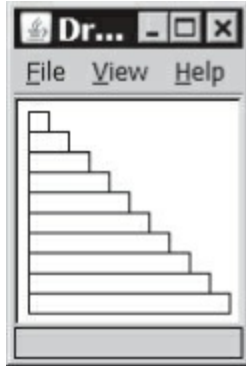


Figure 3G.31 Expected graphical output of Exercise 3G.10

11. Modify your previous Stairs program to draw each of the outputs shown in [Figure 3G.32](#). Modify only the body of your loop. (You may want to make a new table to find the expressions for x, y , *width*, and *height* for each new output.)

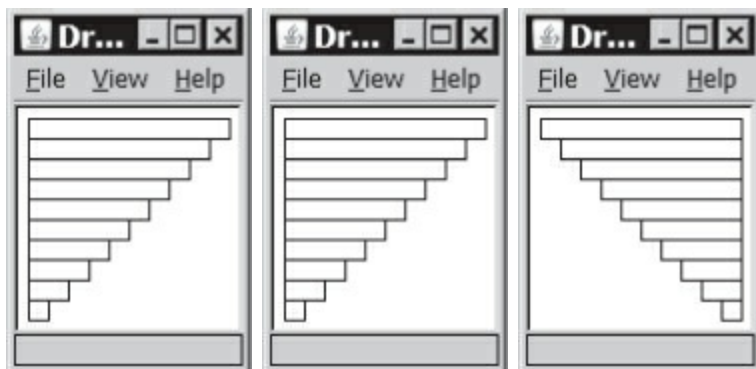


Figure 3G.32 Expected

graphical outputs of Exercise 3G.11

12. Write a program called `Triangle` that uses the `DrawingPanel` to draw the figure shown in [Figure 3G.33](#).

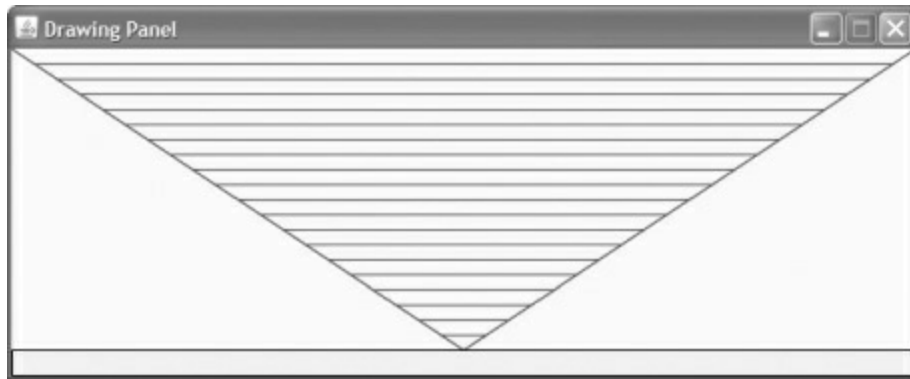


Figure 3G.33 Expected graphical output of Exercise 3G.12

The window is 600×200 pixels in size. The background is yellow and the lines are blue. The lines are 10 pixels apart vertically, and the diagonal lines intersect at the bottom of the figure in its horizontal center.

13. Write a program called `Football` that uses the `DrawingPanel` to draw the figure shown in [Figure 3G.34](#). Though the figure looks to contain curves, it is entirely made of straight lines.

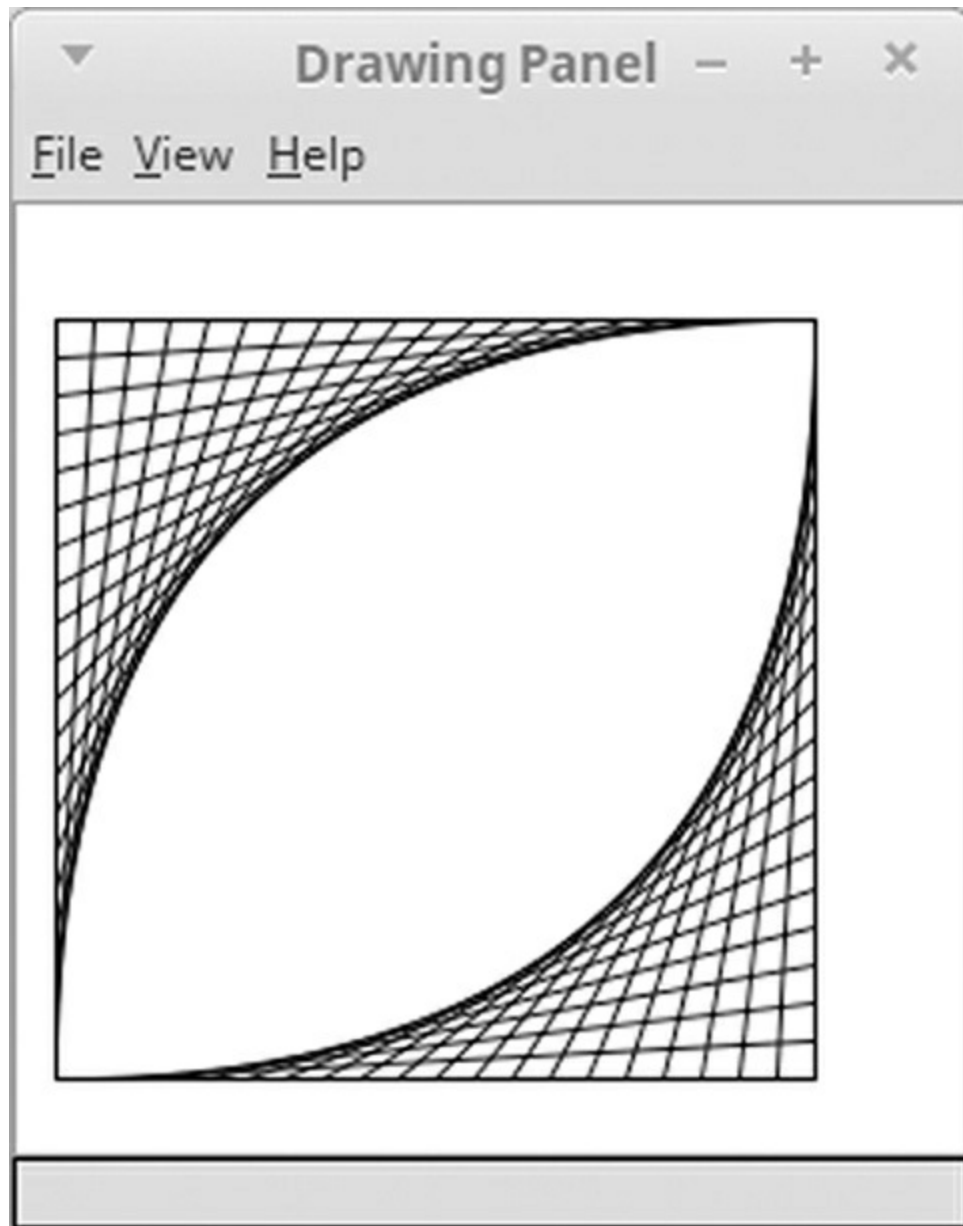


Figure 3G.34 Expected graphical output of Exercise 3G.13

The window is 250 3 250 pixels in size. There is an outer rectangle from (10, 30) to (210, 230), and a set of black lines drawn around the edges

every 10 pixels. For example, along the top-left there is a line from (10, 200) to (20, 30), a line from (10, 190) to (30, 30), a line from (10, 180) to (40, 30), . . . and along the bottom-right there is a line from (20, 210) to (210, 200), a line from (30, 210) to (210, 190), and so on.

Exercises

1. Write a method called `fractionSum` that accepts an integer parameter n and returns as a double the sum of the first n terms of the sequence

$$\sum_{i=1}^n \frac{1}{i}$$

In other words, the method should generate the following sequence:

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \dots$$

You may assume that the parameter n is nonnegative.

2. Write a method called `rep1` that accepts a `String` and a number of repetitions as parameters and returns the `String` concatenated that many times. For example, the call `rep1("hello", 3)` should return `"hellohellohello"`. If the number of repetitions is zero or less, the method should return an empty string.
3. Write a method called `season` that takes as parameters two integers representing a month and day and returns a `String` indicating the season for that month and day. Assume that the month is specified as an integer between 1 and 12 (1 for January, 2 for February, and so on) and that the day of the month is a number between 1 and 31. If the date falls between 12/16 and 3/15, the method should return `"winter"`. If the date falls between 3/16 and 6/15, the method should return `"spring"`. If the date falls between 6/16 and 9/15, the method should return `"summer"`. And if the date falls between 9/16 and 12/15, the method should return `"fall"`.
4. Write a method called `daysInMonth` that takes a month (an integer between 1 and 12) as a parameter and returns the number of days in that month in this year. For example, the call `daysInMonth(9)` would return 30 because September has 30 days. Assume that the code is not being run during a leap year (that February always has 28 days). The following table lists the number of days in each month:

Month	1	2	3	4	5	6	7	8	9	10	11	12
	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Days	31	28	31	30	31	30	31	31	30	31	30	31

- Write a method called `pow` that accepts a base and an exponent as parameters and returns the base raised to the given power. For example, the call `pow(3, 4)` should return $3 * 3 * 3 * 3$, or 81. Assume that the base and exponent are nonnegative.
- Write a method called `printRange` that accepts two integers as arguments and prints the sequence of numbers between the two arguments, separated by spaces. Print an increasing sequence if the first argument is smaller than the second; otherwise, print a decreasing sequence. If the two numbers are the same, that number should be printed by itself. Here are some sample calls to `printRange`:

```
printRange(2, 7);
printRange(19, 11);
printRange(5, 5);
```

The output produced from these calls should be the following sequences of numbers:

```
2 3 4 5 6 7
19 18 17 16 15 14 13 12 11
5
```

- Write a static method called `xo` that accepts an integer *size* as a parameter and prints a square of *size* by *size* characters, where all characters are “o” except that an “x” pattern of “x” characters has been drawn from the corners of the square. On the first line, the first and last characters are “x”; on the second line, the second and second-from-last characters are “x”; and so on. Here are two example outputs:

```
xo(5); xo(6);
x000x  x0000x
0x0x0  0x00x0
00x00  00xx00
```

```
0X0X0 00XX00
X000X 0X00X0
      X0000X
```

8. Write a method called `smallestLargest` that accepts a `Scanner` for the console as a parameter and asks the user to enter numbers, then prints the smallest and largest of all the numbers supplied by the user. You may assume that the user enters a valid number greater than 0 for the number of numbers to read. Here is a sample execution:

```
How many numbers do you want to enter? 4
Number 1: 5
Number 2: 11
Number 3: 22
Number 4: 3
Smallest = 22
Largest = 11
```

9. Write a method called `evenSumMax` that accepts a `Scanner` for the console as a parameter. The method should prompt the user for a number of integers, then prompt the integer that many times. Once the user has entered all the integers, the method should print the sum of all the even numbers the user typed, along with the largest even number typed. You may assume that the user will type at least one nonnegative even integer. Here is an example dialogue:

```
How many integers? 4
Next integer? 2
Next integer? 9
Next integer? 18
Next integer? 4
Even sum = 24, Even max = 18
```

10. Write a method called `printGPA` that accepts a `Scanner` for the console as a parameter and calculates a student's grade point average. The user will type a line of input containing the student's name, then a number that represents the number of scores, followed by that many integer scores. Here are two example dialogues:

Enter a student record: **Maria 5 72 91 84 89 78**

Maria's grade is 82.8

Enter a student record: **Jordan 4 86 71 62 90**

Jordan's grade is 77.25

Maria's grade is 82.8 because her average of $(72 + 91 + 84 +$

11. Write a method called `longestName` that accepts a `Scanner` for the console and an integer n as parameters and prompts for n names, then prints the longest name (the name that contains the most characters) in the format shown below, which might result from a call of `longestName(console, 4)`:

```
name #1? Roy
name #2? DANE
name #3? sTeFaNiE
name #4? Mariana
Stefanie's name is longest
```

12. Write the method called `printTriangleType` referred to in Self-Check Problem 25. This method accepts three integer arguments representing the lengths of the sides of a triangle and prints the type of triangle that these sides form. Here are some sample calls to `printTriangleType`:

```
printTriangleType(5, 7, 7);
printTriangleType(6, 6, 6);
printTriangleType(5, 7, 8);
printTriangleType(2, 18, 2);
```

The output produced by these calls should be

```
isosceles
equilateral
scalene
isosceles
```

Your method should throw an `IllegalArgumentException` if passed invalid values, such as ones where one side's length is longer than the sum of the other two, which is impossible in a triangle. For example, the call of `printTriangleType(2, 18, 2)`; should throw an exception.

13. Write a method called `average` that takes two integers as parameters and returns the average of the two integers.
14. Modify your `pow` method from Exercise 5 to make a new method called `pow2` that uses the type `double` for the first parameter and that works correctly for negative numbers. For example, the call `pow2(-4.0, 3)` should return $-4.0 * -4.0 * -4.0$, or -64.0 , and the call `pow2(4.0, -2)` should return $1 / 16$, or 0.0625 .
15. Write a method called `getGrade` that accepts an integer representing a student's grade in a course and returns that student's numerical course grade. The grade can be between 0.0 (failing) and 4.0 (perfect). Assume that scores are in the range of 0 to 100 and that grades are based on the following scale:

Score Grade

< 60 0.0

60–62 0.7

63 0.8

64 0.9

65 1.0

...

92 3.7

93 3.8

94 3.9

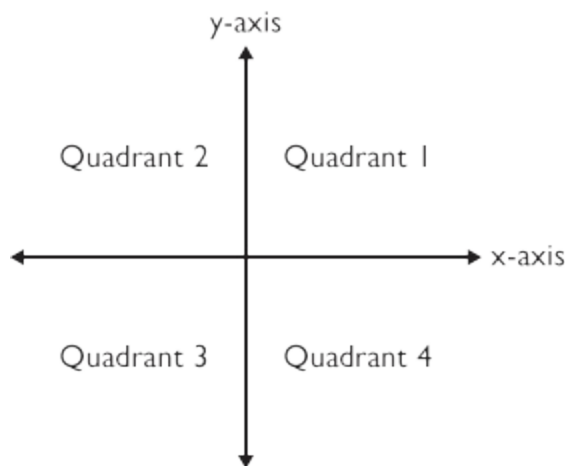
>= 95 4.0

For an added challenge, make your method throw an `IllegalArgumentException` if the user passes a grade lower than 0 or higher than 100 .

16. Write a method called `printPalindrome` that accepts a `Scanner` for the console as a parameter, prompts the user to enter one or more words, and prints whether the entered `String` is a palindrome (i.e., reads the same forward as it does backward, like "abba" or "racecar").

For an added challenge, make the code case-insensitive, so that words like “Abba” and “Madam” will be considered palindromes.

17. Write a method called `stutter` that accepts a string parameter and returns that string with its characters repeated twice. For example, `stutter("Hello!")` returns `"HHee1111loo!!"`
18. Write a method called `wordCount` that accepts a `String` as its parameter and returns the number of words in the `String`. A word is a sequence of one or more nonspace characters (any character other than ' '). For example, the call `wordCount("hello")` should return 1, the call `wordCount("how are you?")` should return 3, the call `wordCount("this string has wide spaces ")` should return 5, and the call `wordCount(" ")` should return 0.
19. Write a method called `quadrant` that accepts as parameters a pair of `double` values representing an (x, y) point and returns the quadrant number for that point. Recall that quadrants are numbered as integers from 1 to 4 with the upper-right quadrant numbered 1 and the subsequent quadrants numbered in a counterclockwise fashion:



Notice that the quadrant is determined by whether the x and y coordinates are positive or negative numbers. Return 0 if the point lies on the x - or y -axis. For example, the call of `quadrant(-2.3, 3.5)` should return 2 and the call of `quadrant(4.5, -4.5)` should return 4.

20. Write a method called `numUnique` that takes three integers as parameters

and returns the number of unique integers among the three. For example, the call `numUnique(18, 3, 4)` should return 3 because the parameters have three different values. By contrast, the call `numUnique(6, 7, 6)` should return 2 because there are only two unique numbers among the three parameters: 6 and 7.

21. Write a method called `perfectNumbers` that accepts an integer maximum as its parameter and prints all “perfect numbers” up to and including that maximum. A perfect number is an integer that is equal to the sum of its proper factors, that is, all numbers that evenly divide it other than 1 and itself. For example, 28 is a perfect number because $1 + 2 + 4 + 7 + 14 = 28$. The call `perfectNumbers(500);` should produce the following output:

Perfect numbers up to 500: 6 28 496

Exercises

1. Write a method called `showTwos` that shows the factors of 2 in a given integer. For example, consider the following calls:

```
showTwos(7);  
showTwos(18);  
showTwos(68);  
showTwos(120);
```

These calls should produce the following output:

```
7 = 7  
18 = 2 * 9  
68 = 2 * 2 * 17  
120 = 2 * 2 * 2 * 15
```

2. Write a method called `gcd` that accepts two integers as parameters and returns the greatest common divisor (GCD) of the two numbers. The GCD of two integers *a* and *b* is the largest integer that is a factor of both *a* and *b*.

One efficient way to compute the GCD of two numbers is to use Euclid's algorithm, which states the following:

$\text{GCD}(a,b)=\text{GCD}(b,a\%b)$
 $\text{GCD}(a, 0)=\text{Absolute value of } a$

3. Write a method called `toBinary` that accepts an integer as a parameter and returns a `String` containing that integer's binary representation. For example, the call of `printBinary(44)` should return "101100".
4. Write a method called `randomX` that prints a lines that contain a random number of "x" characters (between 5 and 20 inclusive) until it prints a line that contains 16 or more characters. For example, the output might look like the following:

```
xxxxxxx
xxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxx
xxxxxxx
xxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxx
```

5. Write a method called `randomLines` that prints between 5 and 10 random strings of letters (between “a” and “z”), one per line. Each string should have random length of up to 80 characters.
6. Write a method called `makeGuesses` that guesses numbers between 1 and 50 inclusive until it makes a guess of at least 48. It should report each guess and at the end should report the total number of guesses made. Here is a sample execution:

```
guess = 43
guess = 47
guess = 45
guess = 27
guess = 49
total guesses = 5
```

7. Write a method called `diceSum` that accepts a `Scanner` for the console as a parameter and prompts for a desired sum, then repeatedly simulates the rolling of 2 six-sided dice until their sum is the desired sum. Here is a sample dialogue with the user:

```
Desired dice sum: 9
4 and 3 = 7
3 and 5 = 8
5 and 6 = 11
5 and 6 = 11
1 and 5 = 6
6 and 3 = 9
```

8. Write a method called `randomWalk` that performs steps of a random one-dimensional walk. The random walk should begin at position 0. On each step, you should either increase or decrease the position by 1 (each with

equal probability). Your code should continue making steps until a position of 3 or 23 is reached, and then report the maximum position that was reached during the walk. The output should look like the following:

```
position = 1
position = 0
position = -1
position = -2
position = -1
position = -2
position = -3
max position = 1
```

9. Write a method called `printFactors` that accepts an integer as its parameter and uses a fencepost loop to print the factors of that number, separated by the word "and". For example, the factors of the number 24 should print as the following:

```
1 and 2 and 3 and 4 and 6 and 8 and 12 and 24
```

You may assume that the parameter's value is greater than 0, or you may throw an exception if it is 0 or negative. Your method should print nothing if the empty string ("") is passed.

10. Write a method called `hopscotch` that accepts an integer number of "hops" as its parameter and prints a pattern of numbers that resembles a hopscotch board. A "hop" is a three-number sequence where the output shows two numbers on a line, followed by one number on its own line. 0 hops is a board up to 1; one hop is a board up to 4; two hops is a board up to 7; and so on. For example, the call of `hopscotch(3);` should print the following output:

```
    1
2    3
    4
5    6
    7
8    9
   10
```

A call of `hopscotch(0)`; should print only the number 1. If it is passed a negative value, the method should produce no output.

11. Write a method called `threeHeads` that repeatedly flips a coin until the results of the coin toss are three heads in a row. You should use a `Random` object to make it equally likely that a head or a tail will appear. Each time the coin is flipped, display H for heads or T for tails. When three heads in a row are flipped, the method should print a congratulatory message. Here is a possible output of a call to the method:

```
T T H T T T H T H T H H H
Three heads in a row!
```

12. Write a method called `printAverage` that uses a sentinel loop to repeatedly prompt the user for numbers. Once the user types any number less than zero, the method should display the average of all nonnegative numbers typed. Display the average as a double. Here is a sample dialogue with the user:

```
Type a number: 7
Type a number: 4
Type a number: 16
Type a number: -4
Average was 9.0
```

If the first number that the user types is negative, do not print an average:

```
Type a number: -2
```

13. Write a method called `consecutive` that accepts three integers as parameters and returns `true` if they are three consecutive numbers—that is, if the numbers can be arranged into an order such that, assuming some integer k , the parameters' values are k , $k + 1$, and $k + 2$. Your method should return `false` if the integers are not consecutive. Note that order is not significant; your method should return the same result for the same three integers passed in any order.

For example, the calls `consecutive(1, 2, 3)`, `consecutive(3, 2, 4)`, and `consecutive(-10, -8, -9)` would return `true`. The calls `consecutive(3, 5, 7)`, `consecutive(1, 2, 2)`, and `consecutive(7, 7, 9)` would return `false`.

14. Write a method called `hasMidpoint` that accepts three integers as parameters and returns `true` if one of the integers is the midpoint between the other two integers; that is, if one integer is exactly halfway between them. Your method should return `false` if no such midpoint relationship exists. For example, the call `hasMidpoint(7, 4, 10)` should return `true` because 7 is halfway between 4 and 10. By contrast, the call `hasMidpoint(9, 15, 8)` should return `false` because no integer is halfway between the other two. The integers could be passed in any order; the midpoint could be the 1st, 2nd, or 3rd. You must check all cases. If your method is passed three of the same value, return `true`.
15. Write a method called `dominant` that accepts three integers as parameters and returns `true` if any one of the three integers is larger than the sum of the other two integers. The integers might be passed in any order, so the largest value could be any of the three. For example, the call `dominant(4, 9, 2)` returns `true` because 9 is larger than 4 + 2. Assume that none of the numbers is negative.
16. Write a method called `anglePairs` that accepts three angles (integers), measured in degrees, as parameters and returns whether or not there exist both complementary and supplementary angles among the three angles passed. Two angles are complementary if their sum is exactly 90 degrees; two angles are supplementary if their sum is exactly 180 degrees. Therefore, the method should return `true` if any two of the three angles add up to 90 degrees and also any two of the three angles add up to 180 degrees. For example, the call `anglePairs(120, 60, 30)` returns `true`. Assume that each angle passed is nonnegative.
17. Write a method called `monthApart` that accepts four integer parameters, *m1*, *d1*, *m2*, and *d2*, representing two calendar dates. Each date consists of a month (1 through 12) and a day (1 through the number of days in that month [28–31]). Assume that all parameter values passed are valid. The method should return `true` if the dates are at least a month apart and

false otherwise. For example, the call of `monthApart(4, 15, 5, 22)` would return true while the call of `monthApart(9, 19, 10, 17)` would return false. Assume that all the dates in this problem occur during the same year. Note that the first date could come before or after the second date.

18. Write a method called `digitSum` that accepts an integer as a parameter and returns the sum of the digits of that number. For example, the call `digitSum(29107)` returns 2 1 9 1 1 1 0 1 7 or 19. For negative numbers, return the same value that would result if the number were positive. For example, `digitSum(-456)` returns 4 1 5 1 6 or 15. The call `digitSum(0)` returns 0.
19. Write a method called `firstDigit` that returns the first (most significant) digit of an integer. For example, `firstDigit(3572)` should return 3. It should work for negative numbers as well; `firstDigit(-947)` should return 9.
20. Write a method called `digitRange` that accepts an integer as a parameter and returns the range of values of its digits. The range is defined as 1 more than the difference between the largest and smallest digit value. For example, the call of `digitRange(68437)` would return 6 because the largest digit value is 8 and the smallest is 3, so 8 2 3 1 1 5 6. If the number contains only one digit, return 1. You should solve this problem without using a `String`.
21. Write a method called `swapDigitPairs` that accepts an integer n as a parameter and returns a new integer whose value is similar to n 's but with each pair of digits swapped in order. For example, the call of `swapDigitPairs(482596)` would return 845269. Notice that the 9 and 6 are swapped, as are the 2 and 5, and the 4 and 8. If the number contains an odd number of digits, leave the leftmost digit in its original place. For example, the call of `swapDigitPairs(1234567)` would return 1325476. You should solve this problem without using a `String`.
22. Write a method called `allDigitsOdd` that returns whether every digit of a positive integer is odd. Return true if the number consists entirely of odd digits (1, 3, 5, 7, 9) and false if any of its digits are even (0, 2, 4, 6,

8). For example, the call `allDigitsOdd(135319)` returns `true` but `allDigitsOdd(9145293)` returns `false`.

23. Write a method called `hasAnOddDigit` that returns whether a given positive integer has at least one digit whose value is odd. Return `true` if the number has at least one odd digit and `false` if none of its digits are odd. For example, the call `hasAnOddDigit(4822116)` should return `true` and `hasAnOddDigit(2448)` should return `false`.
24. Write a method called `isAllVowels` that returns whether a string consists entirely of vowels (a, e, i, o, or u, case-insensitively). If and only if every character of the string is a vowel, your method should return `true`. For example, the call `isAllVowels("eIEiO")` returns `true` and `isAllVowels("oink")` returns `false`. You should return `true` if passed the empty string, since it does not contain any non-vowel characters.

Exercises

1. Write a method called `boyGirl` that accepts a Scanner that is reading its input from a file containing a series of names followed by integers. The names alternate between boys' names and girls' names. Your method should compute the absolute difference between the sum of the boys' integers and the sum of the girls' integers. The input could end with either a boy or girl; you may not assume that it contains an even number of names. For example, if the input file contains the following text:

```
Erik 3 Rita 7 Tanner 14 Jillyn 13 Curtis 4 Stefanie 12 Ben 6
```

Then the method should produce the following console output, since the boys' sum is 27 and the girls' sum is 32:

```
4 boys, 3 girls  
Difference between boys' and girls' sums: 5
```

2. Write a method called `evenNumbers` that accepts a Scanner reading input from a file with a series of integers, and report various statistics about the integers to the console. Report the total number of numbers, the sum of the numbers, the count of even numbers and the percent of even numbers. For example, if the input file contains the following text:

```
5 7 2 8 9 10 12 98 7 14 20 22
```

Then the method should produce the following console output:

```
12 numbers, sum = 214  
8 evens (66.67%)
```

3. Write a method called `negativeSum` that accepts a Scanner reading input from a file containing a series of integers, and print a message to the console indicating whether the sum starting from the first number is ever negative. You should also return `true` if a negative sum can be reached

and false if not. For example, suppose the file contains the following text:

```
38 4 19 -27 -15 -3 4 19 38
```

Your method would consider the sum of just one number (38), the first two numbers (38 1 4), the first three numbers (38 1 4 1 19), and so on to the end. None of these sums is negative, so the method would produce the following output and return false:

```
no negative sum
```

If the file instead contains the following numbers:

```
14 7 -10 9 -18 -10 17 42 98
```

The method finds that a negative sum of 28 is reached after adding the first six numbers. It should output the following to the console and return true:

```
sum of -8 after 6 steps
```

4. Write a method called `countCoins` that accepts a `Scanner` representing an input file whose data is a series of pairs of tokens, where each pair begins with an integer and is followed by the type of coin, which will be “pennies” (1 cent each), “nickels” (5 cents each), “dimes” (10 cents each), or “quarters” (25 cents each), case-insensitively. Add up the cash values of all the coins and print the total money. For example, if the input file contains the following text:

```
3 pennies 2 quarters 1 Pennies 23 NiCkeLs 4 DIMES
```

For the input above, your method should produce the following output:

```
Total money: $2.09
```

5. Write a method called `collapseSpaces` that accepts a `Scanner` representing an input file as its parameter, then reads that file and outputs it with all its tokens separated by single spaces, collapsing any

sequences of multiple spaces into single spaces. For example, consider the following text:

```
many    spaces    on    this    line!
```

If this text were a line in the file, the same line should be output as follows:

```
many spaces on this line!
```

6. Write a method called `readEntireFile` that accepts a `Scanner` representing an input file as its parameter, then reads that file and returns its entire text contents as a `String`.
7. Write a method called `flipLines` that accepts a `Scanner` for an input file and writes to the console the same file's contents with each pair of lines reversed in order. If the file contains an odd number of lines, leave the last line unmodified. For example, if the file contains:

```
Twas brillig and the slithy toves  
did gyre and gimble in the wabe.  
All mimsey were the borogroves,  
and the mome raths outgrabe.
```

your method should produce the following output:

```
did gyre and gimble in the wabe.  
Twas brillig and the slithy toves  
and the mome raths outgrabe.  
All mimsey were the borogroves,
```

8. Write a method called `doubleSpace` that accepts a `Scanner` for an input file and a `PrintStream` for an output file as its parameters, writing into the output file a double-spaced version of the text in the input file. You can achieve this task by inserting a blank line between each line of output.
9. Write a method called `wordwrap` that accepts a `Scanner` representing an input file as its parameter and outputs each line of the file to the console,

word-wrapping all lines that are longer than 60 characters. For example, if a line contains 112 characters, the method should replace it with two lines: one containing the first 60 characters and another containing the final 52 characters. A line containing 217 characters should be wrapped into four lines: three of length 60 and a final line of length 37.

10. Modify the preceding `wordwrap` method so that it outputs the newly wrapped text back into the original file. (Be careful—don't output into a file while you are reading it!) Also, modify it to use a class constant for the maximum line length rather than hard-coding 60.
11. Modify the preceding `wordwrap` method so that it only wraps whole words, never chopping a word in half. Assume that a word is any whitespace-separated token and that all words are under 60 characters in length.
12. Write a method called `stripHtmlTags` that accepts a `Scanner` representing an input file containing an HTML web page as its parameter, then reads that file and prints the file's text with all HTML tags removed. A tag is any text between the characters `<` and `>`. For example, consider the following text:

```
<html>
<head>
<title>My web page</title>
</head>
<body>
<p>There are many pictures of my cat here,
as well as my <b>very cool</b> blog page,
which contains <font color="red">awesome
stuff about my trip to Vegas.</p>
Here's my cat now:
</body>
</html>
```

If the file contained these lines, your program should output the following text:

My web page

There are many pictures of my cat here,
as well as my very cool blog page,
which contains awesome
stuff about my trip to Vegas.
Here's my cat now:

You may assume that the file is a well-formed HTML document and that there are no < or > characters inside tags.

13. Write a method called `stripComments` that accepts a `Scanner` representing an input file containing a Java program as its parameter, reads that file, and then prints the file's text with all comments removed. A comment is any text on a line from `//` to the end of the line, and any text between `/*` and `*/` characters. For example, consider the following text:

```
import java.util.*;
/* My program
by Suzy Student */
public class Program {
    public static void main(String[] args) {
        System.out.println("Hello, world!"); // a println
    }
    public static /* Hello there */ void foo() {
        System.out.println("Goodbye!"); // comment here
    } /* */
}
```

If the file contained this text, your program should output the following text:

```
import java.util.*;
public class Program {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
    public static void foo() {
        System.out.println("Goodbye!");
    }
}
```

14. Write a method called `printDuplicates` that takes as a parameter a

Scanner containing a series of lines. Your method should examine each line looking for consecutive occurrences of the same token on the same line and print each duplicated token, along with the number of times that it appears consecutively. Nonrepeated tokens are not printed. You may ignore the case of repetition across multiple lines (such as if a line ends with a given token and the next line starts with the same token). You may assume that each line of the file contains at least 1 token of input. For example, consider the following input:

```
hello how how are you you you you
I I I am Jack's Jack's smirking smirking smirking smirking re
bow wow wow yippee yippee yo yippee yippee yay yay yay
one fish two fish red fish blue fish
It's the Muppet Show, wakka wakka wakka
```

Your method should produce the following output:

```
how*2 you*4
I*3 Jack's*2 smirking*4
wow*2 yippee*2 yippee*2 yay*3
wakka*3
```

15. Write a method called `coinFlip` that accepts a Scanner representing an input file of coin flips that are heads (H) or tails (T). Consider each line to be a separate set of coin flips and output the number and percentage of heads in that line. If it is more than 50%, print "You win!". Consider the following file:

```
H T H H T
T t    t T h H
```

For the input above, your method should produce the following output:

```
3 heads (60.0%)
You win!
2 heads (33.3%)
```

16. Write a method called `mostCommonNames` that accepts a Scanner

representing an input file with names on each line separated by spaces. Some names appear multiple times in a row on the same line. For each line, print the most commonly occurring name. If there's a tie, use the first name that had that many occurrences; if all names are unique, print the first name on the line. For example, if the file has this input:

```
Benson Eric   Eric   Kim   Kim Kim   Jenny   Nancy Nancy   Paul  
Ethan   Jamie Jamie Alyssa Alyssa Helene   Helene Jessica Jessi
```

For the input above, your method should produce the following output:

```
Most common: Kim  
Most common: Jamie
```

17. Write a method called `inputStats` that accepts a `Scanner` representing an input file and reports the number of lines, the longest line, the number of tokens on each line, and the length of the longest token on each line. If the file contains the following text:

```
Beware the Jabberwock, my son,  
the jaws that bite, the claws that catch,  
Beware the JubJub bird and shun  
the frumious bandersnatch.
```

For the input above, your method should produce the following output:

```
Line 1 has 5 tokens (longest = 11)  
Line 2 has 8 tokens (longest = 6)  
Line 3 has 6 tokens (longest = 6)  
Line 4 has 3 tokens (longest = 13)  
Longest line: the jaws that bite, the claws that catch,
```

18. Write a method called `plusScores` that accepts a `Scanner` representing an input file containing a series of lines that represent student records. Each student record takes up two lines of input. The first line has the student's name and the second line has a series of plus and minus characters. Below is a sample input:

```
Kane, Erica
--+-+
Chandler, Adam
++-+
Martin, Jake
++++++
```

For each student you should produce a line of output with the student's name followed by a colon followed by the percent of plus characters. For the input above, your method should produce the following output:

```
Kane, Erica: 40.0% plus
Chandler, Adam: 75.0% plus
Martin, Jake: 100.0% plus
```

19. Write a method called `leetSpeak` that accepts two parameters: a `Scanner` representing an input file, and a `PrintStream` representing an output file. Convert the input file's text to "leet speak," where various letters are replaced by other letters/numbers, and output the new text to the given output file. Replace "o" with "0", "l" (lowercase "L") with "1" (the number one), "e" with "3", "a" with "4", "t" with "7", and an "s" at the end of a word with "z". Preserve the original line breaks from the input. Also wrap each word of input in parentheses. For example, if the input file contains the following text:

```
four score and
seven years ago our
fathers brought forth on this continent
a new nation
```

For the input above, your method should produce the following in the output file:

```
(f0ur) (sc0r3) (4nd)
(s3v3n) (y34rZ) (4g0) (0ur)
(f47h3rZ) (br0ugh7) (f0r7h) (0n) (7hiZ) (c0n7in3n7)
(4) (n3w) (n47i0n)
```

Exercises

1. Write a method called `lastIndexOf` that accepts an array of integers and an integer value as its parameters and returns the last index at which the value occurs in the array. The method should return `-1` if the value is not found. For example, in the array `[74, 85, 102, 99, 101, 85, 56]`, the last index of the value `85` is `5`.
2. Write a method called `range` that returns the range of values in an array of integers. The range is defined as 1 more than the difference between the maximum and minimum values in the array. For example, if an array called `list` contains the values `[36, 12, 25, 19, 46, 31, 22]`, the call of `range(list)` should return `35` (`46 - 12 + 1`). You may assume that the array has at least one element.
3. Write a method called `countInRange` that accepts an array of integers, a minimum value, and a maximum value as parameters and returns the count of how many elements from the array fall between the minimum and maximum (inclusive). For example, in the array `[14, 1, 22, 17, 36, 7, -43, 5]`, for minimum value `4` and maximum value `17`, there are four elements whose values fall between `4` and `17`.
4. Write a method called `isSorted` that accepts an array of real numbers as a parameter and returns `true` if the list is in sorted (nondecreasing) order and `false` otherwise. For example, if arrays named `list1` and `list2` store `[16.1, 12.3, 22.2, 14.4]` and `[1.5, 4.3, 7.0, 19.5, 25.1, 46.2]` respectively, the calls `isSorted(list1)` and `isSorted(list2)` should return `false` and `true` respectively. Assume the array has at least one element. A one-element array is considered to be sorted.
5. Write a method called `mode` that returns the most frequently occurring element of an array of integers. Assume that the array has at least one element and that every element in the array has a value between `0` and `100` inclusive. Break ties by choosing the lower value. For example, if the array passed contains the values `[27, 15, 15, 11, 27]`, your

method should return 15. (*Hint:* You may wish to look at the `Tally` program from this chapter to get an idea how to solve this problem.) Can you write a version of this method that does not rely on the values being between 0 and 100?

6. Write a method called `stdev` that returns the standard deviation of an array of integers. Standard deviation is computed by taking the square root of the sum of the squares of the differences between each element and the mean, divided by one less than the number of elements. (It's just that simple!) More concisely and mathematically, the standard deviation of an array a is written as follows:

$$\text{stdev}(a) = \sqrt{\sum_{i=0}^{a.\text{length}-1} (a[i] - \text{average}(a))^2 / a.\text{length} - 1}$$

For example, if the array passed contains the values [1, -2, 4, -4, 9, -6, 16, -8, 25, -10], your method should return approximately 11.237.

7. Write a method called `kthLargest` that accepts an integer k and an array a as its parameters and returns the element such that k elements have greater or equal value. If $k = 0$, return the largest element; if $k = 1$, return the second-largest element, and so on. For example, if the array passed contains the values [74, 85, 102, 99, 101, 56, 84] and the integer k passed is 2, your method should return 99 because there are two values at least as large as 99 (101 and 102). Assume that $0 \leq k < a.\text{length}$. (*Hint:* Consider sorting the array or a copy of the array first.)
8. Write a method called `median` that accepts an array of integers as its parameter and returns the median of the numbers in the array. The median is the number that appears in the middle of the list if you arrange the elements in order. Assume that the array is of odd size (so that one sole element constitutes the median) and that the numbers in the array are between 0 and 99 inclusive. For example, the median of [5, 2, 4, 17, 55, 4, 3, 26, 18, 2, 17] is 5 and the median of [42, 37, 1, 97, 1, 2, 7, 42, 3, 25, 89, 15, 10, 29, 27] is 25. (*Hint:* You may wish to look at the `Tally` program from earlier in this chapter for ideas.)

9. Write a method called `minGap` that accepts an integer array as a parameter and returns the minimum difference or gap between adjacent values in the array, where the gap is defined as the later value minus the earlier value. For example, in the array `[1, 3, 6, 7, 12]`, the first gap is 2 (`3 - 1`), the second gap is 3 (`6 - 3`), the third gap is 1 (`7 - 6`), and the fourth gap is 5 (`12 - 7`). So your method should return 1 if passed this array. The minimum gap could be a negative number if the list is not in sorted order. If you are passed an array with fewer than two elements, return 0.
10. Write a method called `percentEven` that accepts an array of integers as a parameter and returns the percentage of even numbers in the array as a real number. For example, if the array stores the elements `[6, 2, 9, 11, 3]`, then your method should return `40.0`. If the array contains no even elements or no elements at all, return `0.0`.
11. Write a method called `isUnique` that accepts an array of integers as a parameter and returns a boolean value indicating whether or not the values in the array are unique (`true` for yes, `false` for no). The values in the list are considered unique if there is no pair of values that are equal. For example, if passed an array containing `[3, 8, 12, 2, 9, 17, 43, -8, 46]`, your method should return `true`, but if passed `[4, 7, 3, 9, 12, -47, 3, 74]`, your method should return `false` because the value 3 appears twice.
12. Write a method called `priceIsRight` that mimics the guessing rules from the game show *The Price Is Right*. The method accepts as parameters an array of integers representing the contestants' bids and an integer representing a correct price. The method returns the element in the bids array that is closest in value to the correct price without being larger than that price. For example, if an array called `bids` stores the values `[200, 300, 250, 1, 950, 40]`, the call of `priceIsRight(bids, 280)` should return 250, since 250 is the bid closest to 280 without going over 280. If all bids are larger than the correct price, your method should return -1.
13. Write a method called `longestSortedSequence` that accepts an array of integers as a parameter and returns the length of the longest sorted

(nondecreasing) sequence of integers in the array. For example, in the array [3, 8, 10, 1, 9, 14, -3, 0, 14, 207, 56, 98, 12], the longest sorted sequence in the array has four values in it (the sequence 23, 0, 14, 207), so your method would return 4 if passed this array. Sorted means nondecreasing, so a sequence could contain duplicates. Your method should return 0 if passed an empty array.

14. Write a method called `contains` that accepts two arrays of integers *a1* and *a2* as parameters and that returns a boolean value indicating whether or not the sequence of elements in *a2* appears in *a1* (true for yes, false for no). The sequence must appear consecutively and in the same order. For example, consider the following arrays:

```
int[] list1 = {1, 6, 2, 1, 4, 1, 2, 1, 8};  
int[] list2 = {1, 2, 1};
```

The call of `contains(list1, list2)` should return `true` because the sequence of values in `list2` [1, 2, 1] is contained in `list1` starting at index 5. If `list2` had stored the values [2, 1, 2], the call of `contains(list1, list2)` would return `false`. Any two lists with identical elements are considered to contain each other. Every array contains the empty array, and the empty array does not contain any arrays other than the empty array itself.

15. Write a method called `collapse` that accepts an array of integers as a parameter and returns a new array containing the result of replacing each pair of integers with the sum of that pair. For example, if an array called `list` stores the values [7, 2, 8, 9, 4, 13, 7, 1, 9, 10], then the call of `collapse(list)` should return a new array containing [9, 17, 17, 8, 19]. The first pair from the original list is collapsed into 9 (7 + 2), the second pair is collapsed into 17 (8 + 9), and so on. If the list stores an odd number of elements, the final element is not collapsed. For example, if the list had been [1, 2, 3, 4, 5], then the call would return [3, 7, 5]. Your method should not change the array that is passed as a parameter.
16. Write a method called `append` that accepts two integer arrays as parameters and returns a new array that contains the result of appending

the second array's values at the end of the first array. For example, if arrays `list1` and `list2` store `[2, 4, 6]` and `[1, 2, 3, 4, 5]` respectively, the call of `append(list1, list2)` should return a new array containing `[2, 4, 6, 1, 2, 3, 4, 5]`. If the call instead had been `append(list2, list1)`, the method would return an array containing `[1, 2, 3, 4, 5, 2, 4, 6]`.

17. Write a method called `vowelCount` that accepts a `String` as a parameter and produces and returns an array of integers representing the counts of each vowel in the string. The array returned by your method should hold five elements: the first is the count of As, the second is the count of Es, the third Is, the fourth Os, and the fifth Us. Assume that the string contains no uppercase letters. For example, the call `vowelCount("i think, therefore i am")` should return the array `[1, 3, 3, 1, 0]`.
18. Write a method called `wordLengths` that accepts a `Scanner` for an input file as its parameter. Your method should open the given file, count the number of letters in each token in the file, and output a result diagram of how many words contain each number of letters. For example, consider a file containing the following text:

```
Before sorting:
13 23 480 -18 75
hello how are you feeling today
After sorting:
-18 13 23 75 480
are feeling hello how today you
```

Your method should produce the following output to the console. Use tabs so that the stars line up:

```
1: 0
2: 6  *****
3: 10  *****
4: 0
5: 5  *****
6: 1  *
7: 2  **
8: 2  **
```

Assume that no token in the file is more than 80 characters in length.

19. Write a method called `matrixAdd` that accepts a pair of two-dimensional arrays of integers as parameters, treats the arrays as two-dimensional matrixes, and returns their sum. The sum of two matrixes *A* and *B* is a matrix *C*, where for every row *i* and column *j*, $C_{ij} = A_{ij} + B_{ij}$. You may assume that the arrays passed as parameters have the same dimensions.
20. Write a method called `isMagicSquare` that accepts a two-dimensional array of integers as a parameter and returns `true` if it is a magic square. A square matrix is a *magic square* if all of its row, column, and diagonal sums are equal. For example, $\begin{bmatrix} 2 & 7 & 6 \\ 9 & 5 & 1 \\ 4 & 3 & 8 \end{bmatrix}$ is a square matrix because all eight of the sums are exactly 15.
21. Write a method `grayscale` that converts a color image into a black-and-white image. This is done by averaging the red, green, and blue components of each pixel. For example, if a pixel has RGB values of (red = 100, green = 30, blue = 80), the average of the three components is $(100 + 30 + 80)/3 = 70$, so that pixel becomes (red = 70, green = 70, blue = 70).

Exercises

1. Add the following accessor method to the `Point` class:

```
public int quadrant()
```

Returns which quadrant of the x/y plane the current `Point` object falls in. Quadrant 1 contains all points whose x and y values are both positive. Quadrant 2 contains all points with negative x but positive y . Quadrant 3 contains all points with negative x and y values. Quadrant 4 contains all points with positive x but negative y . If the point lies directly on the x and/or y axis, return 0.

2. Add the following mutator method to the `Point` class:

```
public void flip()
```

Negates and swaps the x/y coordinates of the `Point` object. For example, if an object `pt` initially represents the point (5, 23), after a call of `pt.flip()`; the object should represent (3, 25). If the same object initially represents the point (4, 17), after a call to `pt.flip()`; the object should represent (217, 24).

3. Add the following accessor method to the `Point` class:

```
public int manhattanDistance(Point other)
```

Returns the “Manhattan distance” between the current `Point` object and the given other `Point` object. The Manhattan distance refers to the distance between two places if one can travel between them only by moving horizontally or vertically, as though driving on the streets of Manhattan. In our case, the Manhattan distance is the sum of the absolute values of the differences in their coordinates; in other words, the difference in x plus the difference in y between the points.

4. Add the following accessor method to the `Point` class:

```
public boolean isVertical(Point other)
```

Returns true if the given Point lines up vertically with this Point, that is, if their x-coordinates are the same.

5. Add the following accessor method to the Point class:

```
public double slope(Point other)
```

Returns the slope of the line drawn between this Point and the given other Point. Use the formula $(y_2 - y_1) / (x_2 - x_1)$ to determine the slope between two points (x_1, y_1) and (x_2, y_2) . Note that this formula fails for points with identical x-coordinates, so throw an `IllegalArgumentException` in this case.

6. Add the following accessor method to the Point class:

```
public boolean isCollinear(Point p1, Point p2)
```

Returns whether this Point is collinear with the given two other Points. Points are collinear if a straight line can be drawn that connects them. Two basic examples are three points that have the same x- or y-coordinate. The more general case can be determined by calculating the slope of the line between each pair of points and checking whether this slope is the same for all pairs of points. Use the formula $(y_2 - y_1) / (x_2 - x_1)$ to determine the slope between two points (x_1, y_1) and (x_2, y_2) . (Note that this formula fails for points with identical x-coordinates so this will have to be a special case in your code.) Since Java's double type is imprecise, round all slope values to a reasonable accuracy such as four digits past the decimal point before you compare them.

7. Add the following mutator method to the TimeSpan class:

```
public void add(TimeSpan span)
```

Adds the given amount of time to this time span.

8. Add the following mutator method to the TimeSpan class:

```
public void subtract(TimeSpan span)
```

Subtracts the given amount of time from this time span.

9. Add the following mutator method to the `TimeSpan` class:

```
public void scale(int factor)
```

Scales this time span by the given factor. For example, 1 hour and 45 minutes scaled by 2 equals 3 hours and 30 minutes.

10. Add the following mutator method to the `Stock` class:

```
public void clear()
```

Resets this `Stock`'s number of shares purchased and total cost to 0.

11. Suppose the following `BankAccount` class has been created:

```
1  // Each BankAccount object represents one user's account
2  // information including name and balance of money.
3  public class BankAccount {
4      String name;
5      double balance;
6
7      public void deposit(double amount) {
8          balance = balance + amount;
9      }
10
11     public void withdraw(double amount) {
12         balance = balance - amount;
13     }
14 }
```

Add a field to the `BankAccount` class named `transactionFee` for a real number representing an amount of money to deduct every time the user withdraws money. The default value is \$0.00, but the client can change the value. Deduct the transaction fee money during every `withdraw` call (but not from deposits). Make sure that the balance cannot go negative during a withdrawal. If the withdrawal (amount plus transaction fee) would cause it to become negative, don't modify the balance at all.

12. Add a `toString` method to the `BankAccount` class from the previous exercise. Your method should return a string that contains the account's name and balance separated by a comma and space. For example, if an account object named `yana` has the name "Yana" and a balance of 3.03, the call `yana.toString()` should return the string "Yana, \$3.03".
13. Add a `transfer` method to the `BankAccount` class from the previous exercises. Your method should move money from the current bank account to another account. The method accepts two parameters: a second `BankAccount` to accept the money, and a real number for the amount of money to transfer. There is a \$5.00 fee for transferring money, so this much must be deducted from the current account's balance before any transfer. The method should modify the two `BankAccount` objects such that "this" current object has its balance decreased by the given amount plus the \$5 fee, and the other account's balance is increased by the given amount. If this account object does not have enough money to make the full transfer, transfer whatever money is left after the \$5 fee is deducted. If this account has under \$5 or the amount is 0 or less, no transfer should occur and neither account's state should be modified. The following are some example calls to the method:

```
BankAccount ben = new BankAccount();
ben.deposit(80.00);
BankAccount hal = new BankAccount();
hal.deposit(20.00);
ben.transfer(hal, 20.00);    // ben $55, hal $40 (ben -$25, h
ben.transfer(hal, 10.00);    // ben $40, hal $50 (ben -$15, h
hal.transfer(ben, 60.00);    // ben $85, hal $ 0 (ben +$45, h
```

14. Write a class called `Line` that represents a line segment between two `Points`. Your `Line` objects should have the following methods:

```
public Line(Point p1, Point p2)
```

Constructs a new `Line` that contains the given two `Points`.

```
public Point getP1()
```

Returns this `Line`'s first endpoint.

```
public Point getP2()
```

Returns this Line's second endpoint.

```
public String toString()
```

Returns a String representation of this Line, such as "[(22, 3), (4, 7)]".

15. Add the following accessor method to your Line class:

```
public double getSlope()
```

Returns the slope of this Line. The slope of a line between points (x_1, y_1) and (x_2, y_2) is equal to $(y_2 - y_1) / (x_2 - x_1)$. If x_2 equals x_1 the denominator is zero and the slope is undefined, so you may throw an exception in this case.

16. Add the following constructor to your Line class:

```
public Line(int x1, int y1, int x2, int y2)
```

Constructs a new Line that contains the given two Points.

17. Add the following accessor method to your Line class:

```
public boolean isCollinear(Point p)
```

Returns true if the given Point is collinear with the Points of this Line—that is, if, when this Line is stretched infinitely, it would eventually hit the given Point. Points are collinear if a straight line can be drawn that connects them. Two basic examples are three points that have the same x - or y -coordinate. The more general case can be determined by calculating the slope of the line between each pair of points and checking whether this slope is the same for all pairs of points. Use the formula $(y_2 - y_1) / (x_2 - x_1)$ to determine the slope between two points (x_1, y_1) and (x_2, y_2) . (Note that this formula fails for points with identical x -coordinates, so this will have to be a special case in your code.) Since Java's double type is imprecise, round all slope values to a reasonable

accuracy such as four digits past the decimal point before you compare them.

18. Write a class called `Rectangle` that represents a rectangular two-dimensional region. Your `Rectangle` objects should have the following methods:

```
public Rectangle(int x, int y, int width, int height)
```

Constructs a new `Rectangle` whose top-left corner is specified by the given coordinates and with the given width and height. Throw an `IllegalArgumentException` on a negative width or height.

```
public int getHeight()
```

Returns this `Rectangle`'s height.

```
public int getWidth()
```

Returns this `Rectangle`'s width.

```
public int getX()
```

Returns this `Rectangle`'s x-coordinate.

```
public int getY()
```

Returns this `Rectangle`'s y-coordinate.

```
public String toString()
```

Returns a `String` representation of this `Rectangle`, such as `"Rectangle[x=1,y=2,width=3, height=4]"`.

19. Add the following constructor to your `Rectangle` class:

```
public Rectangle(Point p, int width, int height)
```

Construct a new `Rectangle` whose top-left corner is specified by the given `Point` and with the given width and height.

20. Add the following accessor methods to your Rectangle class:

```
public boolean contains(int x, int y)
public boolean contains(Point p)
```

Returns whether the given Point or coordinates lie inside the bounds of this Rectangle.

21. Add the following method to your Rectangle class:

```
public Rectangle union(Rectangle rect)
```

Returns a new Rectangle that represents the area occupied by the tightest bounding box that contains both this Rectangle and the given other Rectangle.

22. Add the following method to your Rectangle class:

```
public Rectangle intersection(Rectangle rect)
```

Return a new Rectangle that represents the largest rectangular region completely contained within both this Rectangle and the given other Rectangle. If the Rectangles do not intersect at all, returns a Rectangle with width and height both equal to 0.

Exercises

1. Write the class `Marketer` to accompany the other law firm classes described in this chapter. Marketers make \$50,000 (\$10,000 more than general employees) and have an additional method called `advertise` that prints "Act now, while supplies last!" Make sure to interact with the superclass as appropriate.
2. Write a class `Janitor` to accompany the other law firm classes described in this chapter. Janitors work twice as many hours per week as other employees (80 hours/week), they make \$30,000 (\$10,000 less than general employees), they get half as much vacation as other employees (only 5 days), and they have an additional method `clean` that prints "Workin' for the man." Make sure to interact with the superclass as appropriate.
3. Write a class `HarvardLawyer` to accompany the other law firm classes described in this chapter. Harvard lawyers are like normal lawyers, but they make 20% more money than a normal lawyer, they get 3 days more vacation, and they have to fill out four of the lawyer's forms to go on vacation. That is, the `getVacationForm` method should return "pinkpinkpinkpink". Make sure to interact with the superclass as appropriate.
4. Write a class `MonsterTruck` that relates to the `Car` and `Truck` classes from Self-Check Problems 9 and 10 and whose methods have the following behavior. Whenever possible, use inheritance to reuse behavior from the superclasses.

Method	Output/Return
<code>m1</code>	monster 1
<code>m2</code>	truck 1
	car 1
<code>toString</code>	"monster vroomvroom"

5. For the next four problems, consider the task of representing types of tickets to campus events. Each ticket has a unique number and a price. There are three types of tickets: walk-up tickets, advance tickets, and student advance tickets. [Figure 9.10](#) illustrates the types:
- Walk-up tickets are purchased the day of the event and cost \$50.
 - Advance tickets purchased 10 or more days before the event cost \$30, and advance tickets purchased fewer than 10 days before the event cost \$40.
 - Student advance tickets are sold at half the price of normal advance tickets: When they are purchased 10 or more days early they cost \$15, and when they are purchased fewer than 10 days early they cost \$20.

Implement a class called `Ticket` that will serve as the superclass for all three types of tickets. Define all common operations in this class, and specify all differing operations in such a way that every subclass must implement them. No actual objects of type `Ticket` will be created: Each actual ticket will be an object of a subclass type. Define the following operations:

- The ability to construct a ticket by number.
- The ability to ask for a ticket's price.
- The ability to `println` a ticket object as a `String`. An example `String` would be "Number: 17, Price: 50.0".

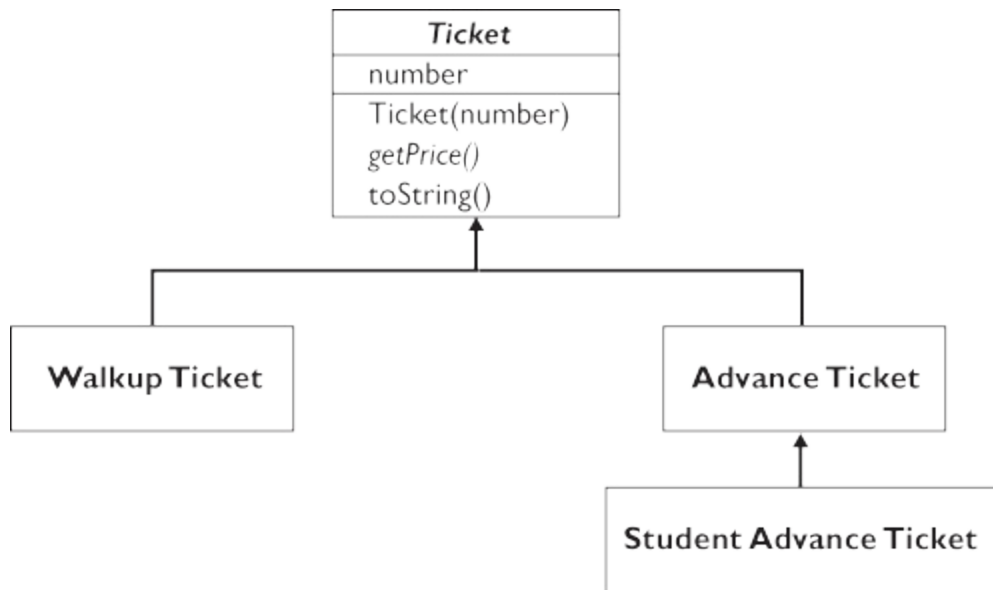


Figure 9.10 Classes of tickets that are available to campus events

6. Implement a class called `walkupTicket` to represent a walk-up event ticket. Walk-up tickets are also constructed by number, and they have a price of \$50.
7. Implement a class called `AdvanceTicket` to represent tickets purchased in advance. An advance ticket is constructed with a ticket number and with the number of days in advance that the ticket was purchased. Advance tickets purchased 10 or more days before the event cost \$30, and advance tickets purchased fewer than 10 days before the event cost \$40.
8. Implement a class called `StudentAdvanceTicket` to represent tickets purchased in advance by students. A student advance ticket is constructed with a ticket number and with the number of days in advance that the ticket was purchased. Student advance tickets purchased 10 or more days before the event cost \$15, and student

advance tickets purchased fewer than 10 days before the event cost \$20 (half of a normal advance ticket). When a student advance ticket is printed, the String should mention that the student must show his or her student ID (for example, "Number: 17, Price: 15.0 (ID required)").

9. MinMaxAccount. A company has written a large class BankAccount with many methods including:

public BankAccount(Startup s)	Constructs a BankAccount object using information in s
public void debit(Debit d)	Records the given debit
public void credit(Credit c)	Records the given credit
public int getBalance()	Returns the current balance in pennies

Design a new class MinMaxAccount whose instances can be used in place of a bank account but include new behavior of remembering the minimum and maximum balances ever recorded for the account. The class should have a constructor that accepts a Startup parameter. The bank account's constructor sets the initial balance on the basis of the startup information. Assume that only debits and credits change an account's balance. Include these new methods in your class:

public int getMin()	Returns the minimum balance in pennies
public int getMax()	Returns the maximum balance in pennies

10. DiscountBill. Suppose a class GroceryBill keeps track of a list of items being purchased at a market:

public GroceryBill(Employee clerk)	Constructs a grocery bill object for the given clerk
public void add(Item i)	Adds the given item to this bill
public double getTotal()	Returns the total cost of these items
public void printReceipt()	Prints a list of items

Grocery bills interact with `Item` objects, each of which has the public methods that follow. A candy bar item might cost 1.35 with a discount of 0.25 for preferred customers, meaning that preferred customers get it for 1.10. (Some items will have no discount, 0.0.) Currently the preceding classes do not consider discounts. Every item in a bill is charged full price, and item discounts are ignored.

```
public double getPrice()    Returns the price for this item
public double getDiscount() Returns the discount for this item
```

Define a class `DiscountBill` that extends `GroceryBill` to compute discounts for preferred customers. Its constructor accepts a parameter for whether the customer should get the discount. Your class should also adjust the total reported for preferred customers. For example, if the total would have been \$80 but a preferred customer is getting \$20 in discounts, then `getTotal` should report the total as \$60 for that customer. Also keep track of the number of items on which a customer is getting a nonzero discount and the sum of these discounts, both as a total amount and as a percentage of the original bill. Include the extra methods that follow, which allow a client to ask about the discount. Return 0.0 if the customer is not a preferred customer or if no items were discounted.

```
public
DiscountBill(Employee clerk, boolean
preferred)    Constructs bill for given clerk

public int
getDiscountCount()    Returns the number of items that were
                        discounted, if any

public double
getDiscountAmount()    Returns the total discount for this list of
                        items, if any

public double
getDiscountPercent()    Returns the percent of the total discount as
                        a percent of what the total would have
                        been otherwise
```

11. `FilteredAccount`. A cash processing company has a class called `Account` used to process transactions:

<code>public</code>	
<code>Account(Client c)</code>	Constructs an account using client information
<code>public boolean</code>	Processes the next transaction, returning true if
<code>process(Transaction t)</code>	the transaction was approved and false otherwise

Account objects interact with Transaction objects, which have many methods including

<code>public int</code>	Returns the value of this transaction in pennies (could be
<code>value()</code>	negative, positive or zero)

Design a new class called `FilteredAccount` whose instances can be used in place of normal accounts but which include the extra behavior of not processing transactions with a value of 0. More specifically, the new class should indicate that a zero-valued transaction was approved but shouldn't call the `process` method for it. Your class should have a single constructor that accepts a parameter of type `Client`, and it should include the following method:

<code>public double</code>	Returns the percent of transactions filtered out
<code>percentFiltered()</code>	(between 0.0 and 100.0); returns 0.0 if no transactions are submitted

12. Add an `equals` method to the `TimeSpan` class introduced in [Chapter 8](#). Two time spans are considered equal if they represent the same number of hours and minutes.
13. Add an `equals` method to the `Cash` class introduced in this chapter. Two cash objects are considered equal if they represent the same amount of money.
14. Add an `equals` method to each of the `Rectangle`, `Circle`, and `Triangle` classes introduced in this chapter. Two shapes are considered equal if their fields have equivalent values.
15. Write a class named `Octagon` whose objects represent regular octagons (eight-sided polygons). Your class should implement the `Shape` interface

defined in this chapter, including methods for its area and perimeter. An octagon object is defined by its side length. (You may need to search online to find formulas for the area and perimeter of a regular octagon.)

16. Write a class named `Hexagon` whose objects represent regular hexagons (6-sided polygons). Your class should implement the `Shape` interface defined in this chapter.
17. Declare an interface called `Incrementable` which represents items that store an integer that can be incremented in some way. The interface has a method called `increment` that increments the value and a method called `getValue` that returns the value. Once you have written the interface, write two classes called `SequentialIncrementer` and `RandomIncrementer` that implement the interface. The `SequentialIncrementer` begins its value at 0 and increases it by 1 each time it is incremented. The `RandomIncrementer` begins its value at a random integer and changes it to a new random integer each time it is incremented.

Exercises

1. Write a method called `averageVowels` that takes an `ArrayList` of strings as a parameter and returns the average number of vowel characters (a, e, i, o, u) in all Strings in the list. If your method is passed an empty `ArrayList`, it should return `0.0`.
2. Write a method called `swapPairs` that switches the order of values in an `ArrayList` of strings in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, then the next two, and so on. If the number of values in the list is odd, the method should not move the final element. For example, if the list initially stores `["to", "be", "or", "not", "to", "be", "hamlet"]`, your method should change the list's contents to `["be", "to", "not", "or", "be", "to", "hamlet"]`.
3. Write a method called `removeEvenLength` that takes an `ArrayList` of strings as a parameter and removes all of the strings of even length from the list.
4. Write a method called `doubleList` that takes an `ArrayList` of strings as a parameter and replaces every string with two of that same string. For example, if the list stores the values `["how", "are", "you?"]` before the method is called, it should store the values `["how", "how", "are", "are", "you?", "you?"]` after the method finishes executing.
5. Write a method called `scaleByK` that takes an `ArrayList` of integers as a parameter and replaces every integer of value k with k copies of itself. For example, if the list stores the values `[4, 1, 2, 0, 3]` before the method is called, it should store the values `[4, 4, 4, 4, 1, 2, 2, 3, 3, 3]` after the method finishes executing. Zeroes and negative numbers should be removed from the list by this method.
6. Write a method called `minToFront` that takes an `ArrayList` of integers as a parameter and moves the minimum value in the list to the front,

otherwise preserving the order of the elements. For example, if a variable called `list` stores `[3, 8, 92, 4, 2, 17, 9]`, the value 2 is the minimum, so your method should modify the list to store the values `[2, 3, 8, 92, 4, 17, 9]`.

7. Write a method called `removeDuplicates` that takes as a parameter a sorted `ArrayList` of strings and eliminates any duplicates from the list. For example, if the list stores the values `["be", "be", "is", "not", "or", "question", "that", "the", "to", "to"]` before the method is called, it should store the values `["be", "is", "not", "or", "question", "that", "the", "to"]` after the method finishes executing. Because the values will be sorted, all of the duplicates will be grouped together. Assume that the `ArrayList` contains only `String` values, but keep in mind that it might be empty.
8. Write a method called `removeZeroes` that takes as a parameter an `ArrayList` of integers and eliminates any occurrences of the number 0 from the list. For example, if the list stores the values `[0, 7, 2, 0, 0, 4, 0]` before the method is called, it should store the values `[7, 2, 4]` after the method finishes executing.
9. Write a method called `rangeBetweenZeroes` that takes as a parameter an `ArrayList` of integers and returns the number of indexes apart the two farthest occurrences of the number 0 are. For example, if the list stores the values `[7, 2, 0, 0, 4, 0, 9, 0, 6, 4, 8]` when the method is called, it should return 6, because the occurrences of 0 that are farthest apart are at indexes 2 and 7, and the range 2 through 7 has six elements. If only one 0 occurs in the list, your method should return 1. If no 0s occur, your method should return 0.
10. Write a method called `removeInRange` that accepts three parameters, an `ArrayList` of strings, a beginning string, and an ending string, and removes from the list any strings that fall alphabetically between the start and end strings. For example, if the method is passed a list containing the elements `["to", "be", "or", "not", "to", "be", "that", "is", "the", "question"]`, "free" as the start `String`, and "rich" as the end `String`, the list's elements should be changed to `["to", "be", "to", "be", "that", "the"]`. The "or", "not", "is",

and "question" should be removed because they occur alphabetically between "free" and "rich". You may assume that the start string alphabetically precedes the ending string.

11. Write a method called `stutter` that accepts an `ArrayList` of strings and an integer k as parameters and that replaces every string with k copies of that string. For example, if the list stores the values `["how", "are", "you?"]` before the method is called and k is 4, it should store the values `["how", "how", "how", "how", "are", "are", "are", "are", "you?", "you?", "you?", "you?"]` after the method finishes executing. If k is 0 or negative, the list should be empty after the call.
12. Write a method called `markLength4` that accepts an `ArrayList` of strings as a parameter and that places a string of four asterisks "****" in front of every string of length 4. For example, suppose that a variable called `list` contains the values `["this", "is", "lots", "of", "fun", "for", "Java", "coders"]`. The call of `markLength4(list);` should change the list to store the values `["****", "this", "is", "****", "lots", "of", "fun", "for", "****", "Java", "coders"]`.
13. Write a method called `reverse3` that accepts an `ArrayList` of integer values as a parameter and reverses each successive sequence of three values in the list. If the list has extra values that are not part of a sequence of three, those values are unchanged. For example, if a list stores values `[3, 8, 19, 42, 7, 26, 19, -8]`, after the call the list should store the values `[19, 8, 3, 26, 7, 42, 19, -8]`. The first sequence of three (3, 8, 19) has been reversed to be (19, 8, 3). The second sequence (42, 7, 26) has been reversed to be (26, 7, 42), and so on. Notice that 19 and -8 are unchanged because they were not part of a sequence of three values.
14. Write a method called `removeShorterStrings` that accepts an `ArrayList` of strings as a parameter and removes from each pair of values the shorter string in the pair. If the list is of odd length, the final element is unchanged. For example, suppose that a list contains `["four", "score", "and", "seven", "years", "ago", "our"]`. In the first pair ("four" and "score") the shorter string is "four". In the second pair ("and" and "seven") the shorter string is "and". In the third

pair ("years" and "ago") the shorter string is "ago". Your method should remove these shorter strings, changing the list to store ["score", "seven", "years", "our"]. If both strings in a pair have the same length, remove the first string in the pair.

15. Write a method called `filterRange` that accepts an `ArrayList` of integers and two integer values *min* and *max* as parameters and removes all elements whose values are in the range *min* through *max* (inclusive). For example, if a variable called `list` stores the values [4, 7, 9, 2, 7, 7, 5, 3, 5, 1, 7, 8, 6, 7], the call of `filterRange(list, 5, 7);` should remove all values between 5 and 7, changing the list to store [4, 9, 2, 3, 1, 8]. If no elements in range *min-max* are found in the list, or if the list is initially empty, the list's contents are unchanged.
16. Write a method called `clump` that accepts an `ArrayList` of strings as a parameter and replaces each pair of strings with a single string that consists of the two original strings in parentheses separated by a space. If the list is of odd length, the final element is unchanged. For example, suppose that a list contains ["four", "score", "and", "seven", "years", "ago", "our"]. Your method should change the list to store ["(four score)", ("and seven"), ("years ago"), "our"].
17. Write a method called `interleave` that accepts two `ArrayLists` of integers *a1* and *a2* as parameters and inserts the elements of *a2* into *a1* at alternating indexes. If the lists are of unequal length, the remaining elements of the longer list are left at the end of *a1*. For example, if *a1* stores [10, 20, 30] and *a2* stores [4, 5, 6, 7, 8], the call of `interleave(a1, a2);` should change *a1* to store [10, 4, 20, 5, 30, 6, 7, 8]. If *a1* had stored [10, 20, 30, 40, 50] and *a2* had stored [6, 7, 8], the call of `interleave(a1, a2);` would change *a1* to store [10, 6, 20, 7, 30, 8, 40, 50].
18. Modify the `Point` class from [Chapter 8](#) so that it defines a natural ordering by implementing the `Comparable` interface. Compare the `Points` by *y*-major order; that is, points with smaller *y*-coordinate values should come before those with higher *y*-coordinate values. Break ties by comparing *x*-coordinate values.

19. Modify the `TimeSpan` class from [Chapter 8](#) to include a `compareTo` method that compares time spans by their length. A time span that represents a shorter amount of time is considered to be “less than” one that represents a longer amount of time. For example, a span of 3 hours and 15 minutes is greater than a span of 1 hour and 40 minutes.
20. Modify the `CalendarDate` class from this chapter to include a `year` field, and modify its `compareTo` method to take years into account when making comparisons. Years take precedence over months, which take precedence over days. For example, July 18, 1995, comes before March 2, 2001.

Exercises

1. Modify the Sieve program developed in [Section 11.1](#) to make two optimizations. First, instead of storing all integers up to the maximum in the `numbers` list, store only 2 and all odd numbers from 3 upward. Second, write code to ensure that if the first number in the `numbers` list ever reaches the square root of the maximum, all remaining values from the `numbers` list are moved into the `primes` list. (Why is this a valid operation?)
2. Write a method called `alternate` that accepts two `Lists` as its parameters and returns a new `List` containing alternating elements from the two lists, in the following order:
 - First element from first list
 - First element from second list
 - Second element from first list
 - Second element from second list
 - Third element from first list
 - Third element from second list
 - ...

If the lists do not contain the same number of elements, the remaining elements from the longer list should be placed consecutively at the end. For example, for a first list of `[1, 2, 3, 4, 5]` and a second list of `[6, 7, 8, 9, 10, 11, 12]`, a call of `alternate(list1, list2)` should return a list containing `[1, 6, 2, 7, 3, 8, 4, 9, 5, 10, 11, 12]`.

3. Write a method called `removeInRange` that accepts four parameters: a `LinkedList`, an element value, a starting index, and an ending index. The

method's behavior is to remove all occurrences of the given element that appear in the list between the starting index (inclusive) and the ending index (exclusive). Other values and occurrences of the given value that appear outside the given index range are not affected.

For example, for the list [0, 0, 2, 0, 4, 0, 6, 0, 8, 0, 10, 0, 12, 0, 14, 0, 16], a call of `removeInRange(list, 0, 5, 13)` should produce the list [0, 0, 2, 0, 4, 6, 8, 10, 12, 0, 14, 0, 16]. Notice that the zeros located at indexes between 5 inclusive and 13 exclusive in the original list (before any modifications were made) have been removed.

4. Write a method called `partition` that accepts a list of integers and an integer value `E` as its parameter, and rearranges (partitions) the list so that all the elements with values less than `E` occur before all elements with values greater than `E`. The exact order of the elements is unimportant, so long as all elements less than `E` appear before all elements greater than `E`. For example, for the linked list [15, 1, 6, 12, -3, 4, 8, 21, 2, 30, -1, 9], one acceptable ordering of the list after a call of `partition(list, 5)` would be [-1, 1, 2, 4, -3, 12, 8, 21, 6, 30, 15, 9]. You may assume that the list contains no duplicates and does not contain the element value `E`.
5. Write a method called `sortAndRemoveDuplicates` that accepts a list of integers as its parameter and rearranges the list's elements into sorted ascending order, as well as removing all duplicate values from the list. For example, the list [7, 4, -9, 4, 15, 8, 27, 7, 11, -5, 32, -9, -9] would become [-9, -5, 4, 7, 8, 11, 15, 27, 32] after a call to your method. Use a Set as part of your solution.
6. Write a method `countUnique` that accepts a list of integers as a parameter and returns the number of unique integer values in the list. Use a set as auxiliary storage to help you solve this problem. For example, if a list contains the values [3, 7, 3, -1, 2, 3, 7, 2, 15, 15], your method should return 5. The empty list contains 0 unique values.
7. Write a method `countCommon` that accepts two lists of integers as

parameters and returns the number of unique integers that occur in both lists. Use one or more sets as storage to help you solve this problem. For example, if one list contains the values [3, 7, 3, -1, 2, 3, 7, 2, 15, 15] and the other list contains the values [-5, 15, 2, -1, 7, 15, 36], your method should return 4 because the elements -1, 2, 7, and 15 occur in both lists.

8. Write a method `maxLength` that accepts a set of strings as a parameter and that returns the length of the longest string in the list. If your method is passed an empty set, it should return 0.
9. Write a method `hasOdd` that accepts a set of integers as a parameter and returns `true` if the set contains at least one odd integer and `false` otherwise. If passed the empty set, your method should return `false`.
10. Write a method `removeEvenLength` that accepts a set of strings as a parameter and that removes all of the strings of even length from the set.
11. Write a method called `symmetricSetDifference` that accepts two `Set`s as parameters and returns a new `Set` containing their symmetric set difference (that is, the set of elements contained in either of the two sets but not in both). For example, the symmetric difference between the sets [1, 4, 7, 9] and [2, 4, 5, 6, 7] is [1, 2, 5, 6, 9].
12. Write a method `contains3` that accepts a list of strings as a parameter and returns `true` if any single string occurs at least 3 times in the list, and `false` otherwise. Use a map as auxiliary storage.
13. Write a method `isUnique` that accepts a map whose keys and values are strings as a parameter and returns `true` if no two keys map to the same value (and `false` if any two or more keys do map to the same value). For example, if the map contains the following key/value pairs, your method would return `true`: {Marty=Stepp, Stuart=Reges, Jessica=Miller, Amanda=Camp, Hal=Perkins}. But calling it on the following map would return `false`, because of two mappings for Perkins and Reges: {Kendrick=Perkins, Stuart=Reges, Jessica=Miller, Bruce=Reges, Hal=Perkins}.

14. Write a method `intersect` that accepts two maps whose keys are strings and whose values are integers as parameters and returns a new map containing only the key/value pairs that exist in both of the parameter maps. In order for a key/value pair to be included in your result, not only do both maps need to contain a mapping for that key, but they need to map it to the same value. For example, if the two maps passed are {Janet=87, Logan=62, Whitaker=46, Alyssa=100, Stefanie=80, Jeff=88, Kim=52, Sylvia=95} and {Logan=62, Kim=52, Whitaker=52, Jeff=88, Stefanie=80, Brian=60, Lisa=83, Sylvia=87}, your method would return the following new map (the order of the key/value pairs does not matter): {Logan=62, Stefanie=80, Jeff=88, Kim=52}.
15. Write a method `maxOccurrences` that accepts a list of integers as a parameter and returns the number of times the most frequently occurring integer (the “mode”) occurs in the list. Solve this problem using a map as auxiliary storage. If the list is empty, return 0.
16. Write a method called `is1to1` that accepts a map whose keys and values are strings as its parameter and returns `true` if no two keys map to the same value. For example, {Marty=206-9024, Hawking=123-4567, Smith=949-0504, Newton=123-4567} should return `false`, but {Marty=206-9024, Hawking=555-1234, Smith=949-0504, Newton=123-4567} should return `true`. The empty map is considered 1-to-1 and returns `true`.
17. Write a method called `subMap` that accepts two maps from strings to strings as its parameters and returns `true` if every key in the first map is also contained in the second map and maps to the same value in the second map. For example, {Smith=949-0504, Marty=206-9024} is a submap of {Marty=206-9024, Hawking=123-4567, Smith=949-0504, Newton=123-4567}. The empty map is a submap of every map.
18. Write a method called `reverse` that accepts a map from strings to strings as a parameter and returns a new map that is the reverse of the original. The reverse of a map is a new map that uses the values from the original as its keys and the keys from the original as its values. Since a map's values need not be unique but its keys must be, you should have each

value map to a set of keys. In other words, if the original map maps keys of type K to values of type V , the new map should map keys of type V to values that are Sets containing elements of type K . For example, the map {42=Marty, 81=Sue, 17=Ed, 31=Dave, 56=Ed, 3=Marty, 29=Ed} has a reverse of {Marty=[42, 3], Sue=[81], Ed=[17, 56, 29], Dave=[31]}. (The order of the keys and values does not matter.)

19. Write a method called `rarest` that accepts a map whose keys are strings and whose values are integers as a parameter and returns the integer value that occurs the fewest times in the map. If there is a tie, return the smaller integer value. If the map is empty, throw an exception.
20. Write a modified version of the vocabulary program developed in [Chapter 10](#) that uses sets rather than `ArrayLists` to store its words. (The program will be noticeably shorter and will run faster!)

Exercises

1. Write a recursive method called `starString` that accepts an integer as a parameter and prints to the console a string of stars (asterisks) that is 2^n (i.e., 2 to the n^{th} power) long. For example,
 - `starString(0)` should print `*` (because $2^0 == 1$)
 - `starString(1)` should print `**` (because $2^1 == 2$)
 - `starString(2)` should print `****` (because $2^2 == 4$)
 - `starString(3)` should print `*****` (because $2^3 == 8$)
 - `starString(4)` should print `*****` (because $2^4 == 16$)

The method should throw an `IllegalArgumentException` if passed a value less than 0.

2. Write a method called `writeNums` that takes an integer n as a parameter and prints to the console the first n integers starting with 1 in sequential order, separated by commas. For example, consider the following calls:

```
writeNums(5);  
System.out.println(); // to complete the line of output  
writeNums(12);  
System.out.println(); // to complete the line of output
```

These calls should produce the following output:

```
1, 2, 3, 4, 5  
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
```

Your method should throw an `IllegalArgumentException` if passed a value less than 1.

3. Write a method called `writeSequence` that accepts an integer n as a

parameter and prints to the console a symmetric sequence of n numbers composed of descending integers that ends in 1, followed by a sequence of ascending integers that begins with 1. The following table indicates the output that should be produced for various values of n :

Method call	Output produced
-----	-----
<code>writeSequence(1);</code>	1
<code>writeSequence(2);</code>	1 1
<code>writeSequence(3);</code>	2 1 2
<code>writeSequence(4);</code>	2 1 1 2
<code>writeSequence(5);</code>	3 2 1 2 3
<code>writeSequence(6);</code>	3 2 1 1 2 3
<code>writeSequence(7);</code>	4 3 2 1 2 3 4
<code>writeSequence(8);</code>	4 3 2 1 1 2 3 4
<code>writeSequence(9);</code>	5 4 3 2 1 2 3 4 5
<code>writeSequence(10);</code>	5 4 3 2 1 1 2 3 4 5

Notice that when n is odd the sequence has a single 1 in the middle, whereas for even values it has two 1s in the middle. Your method should throw an `IllegalArgumentException` if it is passed a value less than 1.

- Write a recursive method called `doubleDigits` that accepts an integer n as a parameter and returns the integer obtained by replacing every digit of n with two of that digit. For example, `doubleDigits(348)` should return 334488. The call `doubleDigits(0)` should return 0. Calling `doubleDigits` on a negative number should return the negation of calling `doubleDigits` on the corresponding positive number; for example, `doubleDigits(-789)` should return -778899.
- Write a recursive method called `writeBinary` that accepts an integer as a parameter and writes its binary representation to the console. For example, `writeBinary(44)` should print 101100.
- Write a recursive method called `writeSquares` that accepts an integer parameter n and prints the first n squares separated by commas, with the odd squares in descending order followed by the even squares in ascending order. For example, `writeSquares(8);` prints the following output:

49, 25, 9, 1, 4, 16, 36, 64

A call of `writeSquares(1)`; prints 1. The method should throw an `IllegalArgumentException` if it is passed a value less than 1.

7. Write a recursive method called `writeChars` that accepts an integer parameter n and that prints out a total of n characters. The middle character of the output should always be an asterisk ("`*`"). If you are asked to write out an even number of characters, then there will be two asterisks in the middle ("`**`"). Before the asterisk(s) you should write out less-than characters ("`<`"). After the asterisk(s) you should write out greater-than characters ("`>`"). Your method should throw an `IllegalArgumentException` if it is passed a value less than 1. For example, the following calls produce the following output:

Method call	Output produced
<code>writeChars(1);</code>	<code>*</code>
<code>writeChars(2);</code>	<code>**</code>
<code>writeChars(3);</code>	<code><*></code>
<code>writeChars(4);</code>	<code><**></code>
<code>writeChars(5);</code>	<code><<*>></code>
<code>writeChars(6);</code>	<code><<**>></code>
<code>writeChars(7);</code>	<code><<<*>>></code>
<code>writeChars(8);</code>	<code><<<**>>></code>

8. Write a recursive method called `multiplyEvens` that returns the product of the first n even integers. For example, `multiplyEvens(1)` returns 2 and `multiplyEvens(4)` returns 384 (because $2 * 4 * 6 * 8 = 384$). The method should throw an `IllegalArgumentException` if it is passed a value less than or equal to 0.
9. Write a recursive method called `sumTo` that accepts an integer parameter n and returns a real number representing the sum of the first n reciprocals. In other words, `sumTo( $n$ )` returns $(1 + 1/2 + 1/3 + 1/4 + \dots + 1/n)$. For example, `sumTo(2)` should return 1.5. The method should return 0.0 if it is passed the value 0 and throw an `IllegalArgumentException` if it is passed a value less than 0.

10. Write a recursive method called `digitMatch` that accepts two nonnegative integers as parameters and that returns the number of digits that match between them. Two digits match if they are equal and have the same position relative to the end of the number (i.e., starting with the ones digit). In other words, the method should compare the last digits of each number, the second-to-last digits of each number, the third-to-last digits of each number, and so forth, counting how many pairs match. For example, for the call of `digitMatch(1072503891, 62530841)`, the method would compare as follows, and return 4 because four of the pairs match (2-2, 5-5, 8-8, and 1-1).

1	0	7	2	5	0	3	8	9	1
		6	2	5	3	0	8	4	1

11. Write a recursive method called `repeat` that accepts a string `s` and an integer `n` as parameters and that returns `s` concatenated together `n` times. For example, `repeat("hello", 3)` returns "hellohellohello", and `repeat("ok", 1)` returns "ok", and `repeat("bye", 0)` returns "". String concatenation is an expensive operation, so for an added challenge try to solve this problem while performing fewer than `n` concatenations.
12. Write a recursive method called `isReverse` that accepts two strings as parameters and returns `true` if the two strings contain the same sequence of characters as each other but in the opposite order (ignoring capitalization), and `false` otherwise. For example, the call of `isReverse("hello", "eLlOh")` would return `true`. The empty string, as well as any one-letter string, is considered to be its own reverse.
13. Write a recursive method called `indexOf` that accepts two strings as parameters and that returns the starting index of the first occurrence of the second string inside the first string (or -1 if not found). For example, the call of `indexOf("Barack Obama", "bam")` would return 8. (Strings already have an `indexOf` method, but you may not call it in your solution.)
14. Write a recursive method called `evenDigits` that accepts an integer parameter and that returns the integer formed by removing the odd digits

from it. For example, `evenDigits(8342116)` returns 8426 and `evenDigits (-34512)` returns -42. If the number is 0 or has no even digits, such as 35159 or 7, return 0. Leading zeros in the result should be ignored.

15. Write a recursive method called `permut` that accepts two integers n and r as parameters and returns the number of unique permutations of r items from a group of n items. For given values of n and r , this value $P(n, r)$ can be computed as follows:

$$P(n,r)=n!(n-r)!$$

For example, `permut(7, 4)` should return 840. It may be helpful to note that `permut(6, 3)` returns 120, or $840 / 7$.

16. The Sierpinski carpet is a fractal that is defined as follows: The construction of the Sierpinski carpet begins with a square. The square is cut into nine congruent subsquares in a 3-by-3 grid, with the central subsquare removed. The same process is then applied recursively to the eight other subsquares. [Figure 12.14](#) shows the first few iterations of the carpet.

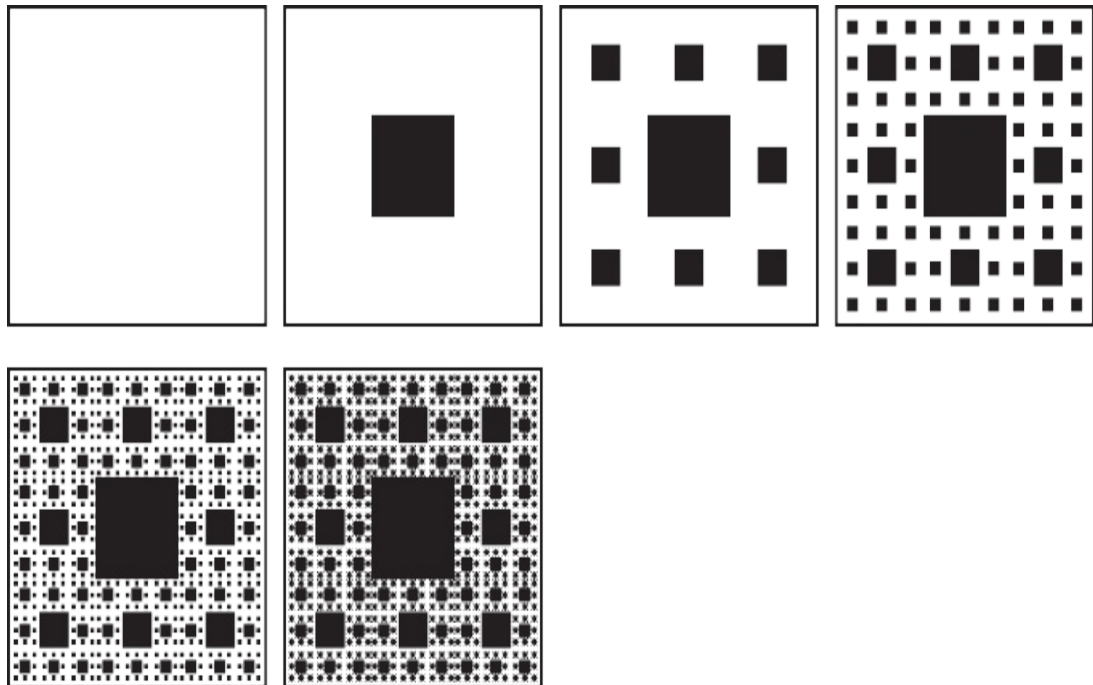


Figure 12.14 Sierpinski carpet

Write a program to draw the carpet on a `DrawingPanel` recursively.

17. The Cantor set is a fractal that is defined by repeatedly removing the middle thirds of line segments as shown in [Figure 12.15](#).



Figure 12.15 Cantor set

Write a program to draw the Cantor set on a `DrawingPanel` recursively.

18. Write a recursive method called `waysToClimb` that takes a positive integer value representing a number of stairs and prints each unique way to climb a staircase of that height, taking strides of one or two stairs at a time. Do not use any loops. Output each way to climb the stairs on its own line, using a 1 to indicate a small stride of 1 stair, and a 2 to indicate a large stride of 2 stairs. The order in which you output the possible ways to climb the stairs is not important, so long as you list the right overall set of ways. For example, the call `waysToClimb(3)`; should produce the following output:

```
[1, 1, 1]
[1, 2]
[2, 1]
```

The call `waysToClimb(4)`; should produce the following output:

```
[1, 1, 1, 1]
[1, 1, 2]
[1, 2, 1]
[2, 1, 1]
```

[2, 2]

19. Write a recursive method called `countBinary` that accepts an integer n as a parameter and that prints all binary numbers that have exactly n digits in ascending order, each on its own line. All n digits should be shown for all numbers, including leading zeros if necessary. Assume that n is nonnegative. If n is 0, a blank line should be produced.

Call	Output
<code>countBinary(1);</code>	0 1
<code>countBinary(2);</code>	00 01 10 11
<code>countBinary(3);</code>	000 001 010 011 100 101 110 111

20. Write a recursive method called `subsets` to find every possible sub-list of a given list. A sub-list of a list L contains 0 or more of L 's elements. Your method should accept a list of strings as its parameter and print every sub-list that could be created from elements of that list, one per line. For example, if the list stores `[Janet, Robert, Morgan, Char]`, the output from your method would be:

```
[Janet, Robert, Morgan, Char]
[Janet, Robert, Morgan]
[Janet, Robert, Char]
[Janet, Robert]
[Janet, Morgan, Char]
[Janet, Morgan]
[Janet, Char]
[Janet]
[Robert, Morgan, Char]
```

```
[Robert, Morgan]
[Robert, Char]
[Robert]
[Morgan, Char]
[Morgan]
[Char]
[]
```

The order in which you show the sub-lists does not matter, and the order of the elements of each sub-list also does not matter. The key thing is that your method should produce the correct overall set of sub-lists as its output. Notice that the empty list is considered one of these sub-lists. You may assume that the list passed to your method is not null and that the list contains no duplicates. Do not use any loops.

21. Write a recursive method called `maxSum` that accepts a list of integers, L , and an integer limit n as parameters and uses backtracking to find the maximum sum that can be generated by adding elements of L that do not exceed n . For example, if you are given the list `[7, 30, 8, 22, 6, 1, 14]` and the limit of 19, the maximum sum that can be generated that does not exceed is 16, achieved by adding 7, 8, and 1. If the list L is empty, or if the limit is not a positive integer, or all of L 's values exceed the limit, return 0.

Each index's element in the list can be added to the sum only once, but the same number value might occur more than once in a list, in which case each occurrence might be added to the sum. For example, if the list is `[6, 2, 1]` you may use up to one 6 in the sum, but if the list is `[6, 2, 6, 1]` you may use up to two sixes.

List L	Limit n	Return Value
<code>[7, 30, 8, 22, 6, 1, 14]</code>	19	16
<code>[5, 30, 15, 13, 8]</code>	42	41
<code>[30, 15, 20]</code>	40	35
<code>[10, 20, 30]</code>	7	0
<code>[10, 20, 30]</code>	20	20
<code>[]</code>	10	0

You may assume that all values in the list are nonnegative. Your method

may alter the contents of the list L as it executes, but L should be restored to its original state before your method returns. Do not use any loops.

22. Write a recursive method called `printSquares` to find all ways to express an integer as a sum of squares of unique positive integers. For example, the call `printSquares(200)`; should produce the following output:

```
1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 8^2 + 9^2
1^2 + 2^2 + 3^2 + 4^2 + 7^2 + 11^2
1^2 + 2^2 + 5^2 + 7^2 + 11^2
1^2 + 3^2 + 4^2 + 5^2 + 6^2 + 7^2 + 8^2
1^2 + 3^2 + 4^2 + 5^2 + 7^2 + 10^2
2^2 + 4^2 + 6^2 + 12^2
2^2 + 14^2
3^2 + 5^2 + 6^2 + 7^2 + 9^2
6^2 + 8^2 + 10^2
```

Some numbers (such as 128 or 0) cannot be represented as a sum of squares, in which case your method should produce no output. Keep in mind that the sum has to be formed with unique integers. Otherwise you could always find a solution by adding 1^2 together until you got to whatever number you are working with.

As with any backtracking problem, this one amounts to a set of choices, one for each integer whose square might or might not be part of your sum. You may generate the choices by doing a for loop over an appropriate range of numbers. Note that the maximum possible integer that can be part of a sum of squares for an integer n is the square root of n .

Exercises

1. Suppose the following array has been declared:

```
// index      0   1   2   3   4   5   6   7   8   9
int[] list = {-2, 8, 13, 22, 25, 25, 38, 42, 51, 103};
```

What indexes will be examined as the middle element by a binary search for each of the following target values? What value will be returned?

- a. 103
- b. 30
- c. 8
- d. -1

2. Suppose the following array has been declared:

```
// index      0   1   2   3   4   5   6   7   8   9  10
int[] numbers = {-1,  3,  5,  8, 15, 18, 22, 39, 40, 42, 50,
```

What indexes will be examined as the middle element by a binary search for each of the following target values? What value will be returned?

- a. 13
- b. 39
- c. 50
- d. 2

3. Suppose the following array has been declared:

```
// index      0   1   2   3   4   5   6   7   8   9  10  11
int[] numbers = {0,  0,  5, 10, 15, 40, 55, 60, 65, 70, 80, 8
```

What indexes will be examined as the middle element by a binary search for each of the following target values? What value will be returned?

- a. 65
- b. 9
- c. 90
- d. 147

4. To which complexity class does the following algorithm belong? Consider N to be the length or size of the array or collection passed to the method. Explain your reasoning.

```
public static int[] mystery1(int[] list) {
    int[] result = new int[2 * list.length];
    for (int i = 0; i < list.length; i++) {
        result[2 * i] = list[i] / 2 + list[i] % 2;
        result[2 * i + 1] = list[i] / 2;
    }
    return result;
}
```

5. To which complexity class does the following algorithm belong?

```
public static void mystery2(int[] list) {
    for (int i = 0; i < list.length / 2; i++) {
        int j = list.length - 1 - i;
        int temp = list[i];
        list[i] = list[j];
        list[j] = temp;
    }
}
```

6. To which complexity class does the following algorithm belong?

```
public static void mystery3(List<String> list) {
    for (int i = 0; i < list.size() - 1; i += 2) {
        String first = list.remove(i);
        list.add(i + 1, first);
    }
}
```

7. To which complexity class does the following algorithm belong?

```
public static void mystery4(List<String> list) {  
    for (int i = 0; i < list.size() - 1; i += 2) {  
        String first = list.get(i);  
        list.set(i, list.get(i + 1));  
        list.set(i + 1, first);  
    }  
}
```

8. Write the state of the elements of each of the following arrays after each pass of the outermost loop of the selection sort algorithm has occurred (after each element is selected and moved into place).

```
int[] numbers1 = {63, 9, 45, 72, 27, 18, 54, 36};  
int[] numbers2 = {37, 29, 19, 48, 23, 55, 74, 12};
```

9. Using the same arrays from the previous problem, trace the complete execution of the merge sort algorithm when called on each array. Show the subarrays that are created by the algorithm and show the merging of subarrays into larger sorted arrays.
10. Write the state of the elements of each of the following arrays after each pass of the outermost loop of the selection sort algorithm has occurred (after each element is selected and moved into place).

```
int[] numbers3 = {8, 5, -9, 14, 0, -1, -7, 3};  
int[] numbers4 = {15, 56, 24, 5, 39, -4, 27, 10};
```

11. Using the same arrays from the previous problem, trace the complete execution of the merge sort algorithm when called on each array. Show the subarrays that are created by the algorithm and show the merging of subarrays into larger sorted arrays.
12. Write the state of the elements of each of the following arrays after each pass of the outermost loop of the selection sort algorithm has occurred (after each element is selected and moved into place).

```
int[] numbers5 = {22, 44, 11, 88, 66, 33, 55, 77};  
int[] numbers6 = {-3, -6, -1, -5, 0, -2, -4, -7};
```

13. Using the same arrays from the previous problem, trace the complete execution of the merge sort algorithm when called on each array. Show the subarrays that are created by the algorithm and show the merging of subarrays into larger sorted arrays.

14. Write code to read a dictionary from a file, then prompt the user for two words and tell the user how many words in the dictionary fall between those two words. Here is a sample run of the program:

```
Type two words: goodbye hello  
There are 4418 words between goodbye and hello
```

Use the binary search algorithm in your solution.

15. Write a Comparator that compares Point objects by their distance from the origin of (0, 0). Points that are closer to the origin are considered to come before those which are further from the origin.
16. Write a Comparator that compares String objects by the number of words they contain. Consider any nonwhitespace string of characters to be a word. For example, “hello” comes before “I see”, which comes before “You can do it”.
17. Write a Comparator that compares String objects of a particular format. Each string is of a form such as "123456 Seattle, WA", beginning with a numeric token that is followed by additional text tokens. Your job is to treat the first tokens as integers and compare them in numerical order. You cannot simply compare them by using the strings' compareTo method, since it would treat the numbers as text and not as integers. For example, "276453 Helena, MT" is greater than "9847 New York, NY". Use a Scanner to tokenize the strings while comparing them.
18. Write a modified version of the selection sort algorithm that selects the largest element each time and moves it to the end of the array, rather than selecting the smallest element and moving it to the beginning. Will this algorithm be faster than the standard selection sort? What will its complexity class (big-Oh) be?
19. Write a modified “dual” version of the selection sort algorithm that

selects both the largest and smallest elements on each pass and moves each of them to the appropriate end of the array. Will this algorithm be faster than the standard selection sort? What predictions would you make about its performance relative to the merge sort algorithm? What will its complexity class (big-Oh) be?

20. Implement an algorithm to shuffle an array of numbers or objects. The algorithm for shuffling should be the following:

```
for (each index i) {  
    choose a random index j where j >= i.  
    swap the elements at indexes i and j.  
}
```

(The constraint about j being greater than or equal to i is actually quite important, if you want your shuffling algorithm to shuffle fairly. Why?)

21. Implement a “bogos” sorting algorithm called *bogo sort* that uses your shuffling algorithm from the previous exercise to sort an array of numbers. The bogo sort algorithm is the following:

```
while (array is not sorted) {  
    shuffle array.  
}
```

Obviously, this is not a very efficient sorting algorithm, but it eventually does shuffle the array into order if you let it run long enough. Try running it on a very small array, such as 8 or 10 elements, to examine its runtime. What is your best guess about the complexity class (big-Oh) of this silly algorithm?

Exercises

Each problem will indicate what kind of structure to use as auxiliary storage. You should not use any other auxiliary data structures to solve the problems, although you can create as many simple variables as you'd like.

It is the authors' intent that you use stacks/queues in stack/queue-like ways only when solving these problems. For example, you should not call index-based methods such as `get`, `search`, or `set` (or use a `for-each` loop) on a stack/queue. You may call only `add`, `remove`, `push`, `pop`, `peek`, `isEmpty`, and `size`. It is also possible to solve all of the exercises without using `peek` if you want an extra challenge.

For problems that accept a stack or queue as a parameter, unless otherwise specified, you should make sure that your method does not damage the state of the parameter. That is, if you modify the parameter stack or queue's elements in your method, you should restore the parameter collection to its original state before your method returns.

Some of these problems have elegant recursive solutions, but the authors' intent is generally that you should not solve these problems recursively, because recursion can circumvent some of the tricky stack/queue manipulation that you are supposed to practice.

1. Write a method called `splitStack` that accepts a stack of integers as a parameter and rearranges its elements so that all the negatives appear on the bottom of the stack and all the nonnegatives appear on the top. If after this method is called you were to `pop` numbers off the stack, you would first get all the nonnegative numbers and then get all the negative numbers. It does not matter what order the numbers appear in as long as all the negatives appear lower in the stack than all the nonnegatives. For example, if the stack stores `[3, 25, 1, 2, 24]`, an acceptable result from your method would be `[25, 24, 3, 1, 2]`. Use a single queue as auxiliary storage.

2. Write a method called `stutter` that accepts a stack of integers as a parameter and replaces every value in the stack with two occurrences of that value. Preserve the original relative order. For example, if the stack stores `[3, 7, 1, 14, 9]`, your method should change it to store `[3, 3, 7, 7, 1, 1, 14, 14, 9, 9]`. Use a single queue as auxiliary storage.
3. Write a method called `copyStack` that accepts a stack of integers as a parameter and returns a copy of the original stack (i.e., a new stack with the same values as the original, stored in the same order as the original). Your method should create the new stack and fill it up with the same values that are stored in the original stack. When your method is done executing, the original stack must be restored to its original state. Use one queue as auxiliary storage.
4. Write a method called `collapse` that accepts a stack of integers as a parameter and that collapses it by replacing each successive pair of integers with the sum of the pair. For example, if the stack stores `[7, 2, 8, 9, 4, 11, 7, 1, 42]`, the first pair should be collapsed into 9 ($7 + 2$), the second pair should be collapsed into 17 ($8 + 9$), and so on. If the stack stores an odd number of elements, such as the 42 at the end of our example stack, the final element is not collapsed. So for this stack your method would yield `[9, 17, 15, 8, 42]`. Use one queue as auxiliary storage.
5. Write a method called `equals` that accepts two stacks of integers as parameters and returns `true` if the two stacks store exactly the same sequence of integer values in the same order. Your method must restore the two stacks to their original state before returning. Use one stack as auxiliary storage.
6. Write a method called `rearrange` that accepts a queue of integers as a parameter and rearranges the order of the values so that all of the even values appear before the odd values and that otherwise preserves the original order of the queue. For example, if the queue stores `[3, 5, 4, 17, 6, 83, 1, 84, 16, 37]`, your method should rearrange it to store `[4, 6, 84, 16, 3, 5, 17, 83, 1, 37]`. Notice that all of the evens appear at the front followed by the odds and that the relative order of the evens and odds is the same as in the original. Use one stack as auxiliary

storage.

7. Write a method called `reverseHalf` that accepts a queue of integers as a parameter and reverses the order of all the elements in odd-numbered positions (position 1, 3, 5, etc.), assuming that the first value in the queue has position 0. For example, if the queue stores [1, 8, 7, 2, 9, 18, 12, 0], your method should change it to store [1, 0, 7, 18, 9, 2, 12, 8]. Notice that numbers in even positions (positions 0, 2, 4, 6) have not moved. That subsequence of integers is still (1, 7, 9, 12). But notice that the numbers in odd positions (positions 1, 3, 5, 7) are now in reverse order relative to the original. In other words, the original subsequence (8, 2, 18, 0) has become (0, 18, 2, 8). Use a single stack as auxiliary storage.
8. Write a method called `isPalindrome` that accepts a queue of integers as a parameter and returns `true` if the numbers in the queue are the same in reverse order. For example, if the queue stores [3, 8, 17, 9, 17, 8, 3], your method should return `true` because this sequence is the same in reverse order. If the queue stores [3, 17, 9, 4, 17, 3], your method would return `false` because this sequence is not the same in reverse order (the 9 and 4 in the middle don't match). The empty queue should be considered a palindrome. Your method must restore the parameter queue to its original state before returning. Use one stack as auxiliary storage.
9. Write a method called `switchPairs` that accepts a stack of integers as a parameter and swaps neighboring pairs of numbers starting at the bottom of the stack. For example, if the stack initially stores [1, 2, 8, 6, 21, 15, 7], your method should swap the first pair (1, 2), the second pair (8, 6), the third pair (21, 15), and so on. If the stack contains an odd number of elements, the element at the top should remain unmodified. So the final state of the stack would be [2, 1, 6, 8, 15, -1, 7]. Use one queue as auxiliary storage.
10. Write a method called `isConsecutive` that accepts a stack of integers as a parameter and that returns `true` if the stack contains a sequence of consecutive integers starting from the bottom of the stack. Consecutive integers are integers that come one after the other, as in 3, 4, 5, etc. If the

stack stores [5, 6, 7, 8, 9, 10], your method should return true. If the stack had instead contained [7, 8, 9, 10, 12], your method should return false because the numbers 10 and 12 are not consecutive. Notice that we look at the numbers starting at the bottom of the stack. Any stack with fewer than two values should be considered to be a list of consecutive integers. Your method must restore the parameter stack to its original state before returning. Use one queue as auxiliary storage.

11. Write a method called `reorder` that accepts a queue of integers as a parameter and that puts the integers into sorted (nondecreasing) order, assuming that the queue is already sorted by absolute value. For example, if the queue stores [1, 2, 22, 4, 25, 8, 28, 12, 215], notice that the values appear in sorted order if you ignore the sign of the numbers. Your method should reorder the values so that the queue stores [215, 28, 25, 22, 1, 2, 4, 8, 12]. Use one stack as auxiliary storage.
12. Write a method called `shift` that accepts a stack of integers and an integer n as parameters and that shifts n values from the bottom of the stack to the top of the stack. For example, if the stack named `s` stores [1, 2, 3, 4, 5, 6, 7, 8], and we make the call `shift(s, 3)`; your method should shift the three values at the bottom of the stack to the top of the stack and leave the other values in the same order, producing [4, 5, 6, 7, 8, 3, 2, 1]. Notice that the value that was at the bottom of the stack is now at the top, the value that was second from the bottom is now second from the top, the value that was third from the bottom is now third from the top, and that the five values not involved in the shift are now at the bottom of the stack in their original order. Use one queue as auxiliary storage. You may assume that the parameter n is ≥ 0 and not larger than the number of elements in the stack.
13. Write a method called `expunge` that accepts a stack of integers as a parameter and makes sure that the stack's elements are in nondecreasing order from top to bottom, by removing from the stack any element that is smaller than any element(s) on top of it. For example, if the stack stores [4, 20, 15, 15, 8, 5, 7, 12, 3, 10, 5, 1], the element values 3, 7, 5, 8, and 4 should be removed because each has an

element above it with a larger value. So your method should change the stack to store [20, 15, 15, 12, 10, 5, 1]. Notice that now the elements are in nondecreasing order from top to bottom. If the stack is empty or has just one element, nothing changes. Use one queue or stack (but not both) as auxiliary storage.

14. Write a method called `reverseFirstK` that accepts an integer k and a queue of integers as parameters and reverses the order of the first k elements of the queue, leaving the other elements in the same relative order. For example, if a queue named `q` stores [10, 20, 30, 40, 50, 60, 70, 80, 90], the call of `reverseFirstK(4, q);` should change the queue to store [40, 30, 20, 10, 50, 60, 70, 80, 90]. If k is 0 or negative, no change should be made. If the queue does not contain at least k elements, your method should throw an `IllegalArgumentException`. Use one queue or stack (but not both) as auxiliary storage.
15. Write a method called `isSorted` that accepts a stack of integers as a parameter and returns `true` if the elements in the stack occur in ascending (nondecreasing) order from top to bottom. That is, the smallest element should be on top, growing larger toward the bottom. For example, if the stack stores [20, 20, 17, 11, 8, 8, 3, 2], your method should return `true`. An empty or one-element stack is considered to be sorted. Your method must restore the parameter stack to its original state before returning. Use one queue or stack (but not both) as auxiliary storage.
16. Write a method called `mirror` that accepts a stack of integers as a parameter and replaces the stack contents with itself plus a mirrored version of itself (the same elements in the opposite order). For example, if the stack stores [10, 53, 19, 24], your method should change it to store [10, 53, 19, 24, 24, 19, 53, 10]. If passed an empty stack, your result should be an empty stack. Use one stack or one queue (but not both) as auxiliary storage to solve this problem.
17. Write a method called `compressDuplicates` that accepts a stack of integers as a parameter and that replaces each sequence of duplicates with a pair of values: a count of the number of duplicates, followed by

the actual duplicated number. For example, if the stack stores [2, 2, 2, 2, 2, 24, 24, 24, 82, 6, 6, 6, 6, 17, 17], your method should change it to store [5, 2, 3, 24, 1, 82, 4, 6, 2, 17]. This new stack indicates that the original had 5 occurrences of 2 at the bottom of the stack followed by 3 occurrences of 24 followed by 1 occurrence of 82, and so on. If the stack is empty, your method should not change it. Use one queue as auxiliary storage.

18. Write a method called `mirrorHalves` that accepts a queue of integers as a parameter and replaces each half of that queue with itself plus a mirrored version of itself (the same elements in the opposite order). For example, if the queue stores [10, 50, 19, 54, 30, 67], your method should change it to store [10, 50, 19, 19, 50, 10, 54, 30, 67, 67, 30, 54]. If your method is passed an empty queue, the result should be an empty queue. If your method is passed a queue whose size is not even, throw an `IllegalArgumentException`. Use one stack or one queue (but not both) as auxiliary storage.
19. Write a method called `removeMin` that accepts a stack of integers as a parameter and removes and returns the smallest value from the stack. For example, if the stack stores [2, 8, 3, 19, 2, 3, 2, 7, 12, 28, 4], your method should remove and return 28, leaving the stack storing [2, 8, 3, 19, 2, 3, 2, 7, 12, 4]. If the minimum value appears more than once, all occurrences of it should be removed. For example, given the same stack, if we again call `removeMin` on it, the method would return 2 and leave the stack storing [8, 3, 19, 3, 7, 12, 4]. Use one queue as auxiliary storage.
20. Write a method called `interleave` that accepts a queue of integers as a parameter and rearranges the elements by alternating the elements from the first half of the queue with those from the second half of the queue. For example, if the queue stores [2, 8, 25, 19, 7, 3, 24, 42], your method should change it to store [2, 7, 8, 3, 25, 24, 19, 42]. To understand the result, consider the two halves of this queue. The first half is [2, 8, 25, 19] and the second half is [7, 3, 24, 42]. These are combined in an alternating fashion to form a sequence of pairs: the first values from each half (2 and 7), then the second values from each

half (8 and 3), and so on. Your method should throw an `IllegalArgumentException` if the queue does not have an even size. Use one stack as auxiliary storage.

Exercises

Each of the following exercises is a method to be added to the `ArrayIntList` class from this chapter.

1. Write a method called `lastIndexOf` that accepts an integer as a parameter and returns the index in the list of the last occurrence of that value, or 21 if the value is not found in the list. For example, if the list stores `[1, 18, 2, 7, 18, 39, 18, 40]`, then the last index of 18 is 6 and the last index of 3 is 21.
2. Write a method called `indexOfSubList` that accepts another list *L* as a parameter and returns the starting index of where *L* first appears in this list, or 21 if it is not found. All elements of *L* must appear in sequence and in the same order. For example, if variables called `list1` and `list2` store `[11, -7, 3, 42, 0, 14]` and `[3, 42, 0]`, respectively, the call of `list1.indexOfSubList(list2)` should return 2.
3. Write a method called `replaceAll` that accepts two integer values as parameters and replaces all occurrences of the first value in the list with the second value. For example, if a variable called `list` stores `[11, -7, 3, 42, 3, 0, 14, 3]`, the call of `list.replaceAll(3, 999);` should change the list to store `[11, -7, 999, 42, 999, 0, 14, 999]`.
4. Write a method called `reverse` that reverses the order of the elements in the array list. For example, if a variable called `list` stores `[11, -7, 3, 42, 0, 14, 56]`, the call of `list.reverse();` should change the list to store `[56, 14, 0, 42, 3, -7, 11]`. An empty or one-element list is not changed by a call to this method.
5. Write a method called `runningTotal` that returns a new `ArrayIntList` that contains a running total of the original list. In other words, the *i*th value in the new list should store the sum of elements 0 through *i* of the original list. For example, given a variable `list` that stores `[2, 3, 5, 4, 7, 15, 20, 7]`, consider what happens when the following call is

made:

```
ArrayList list2 = list.runningTotal();
```

The variable `list2` should store `[2, 5, 10, 14, 21, 36, 56, 63]`.

The original list should not be changed by the method. If the original list is empty, the result should be an empty list. The new list should have the same capacity as the original. Remember that there is a list constructor that accepts a capacity as a parameter.

6. Write a method called `fill` that accepts an integer value as a parameter and replaces every value in the list with that value. For example, if a variable called `list` initially stores `[42, -7, 3, 0, 15]` and the call of `list.fill(2);` is made, the list will be changed to store `[2, 2, 2, 2, 2]`.
7. Write a method called `isPairwiseSorted` that returns whether or not a list of integers is pairwise sorted. A list is considered pairwise sorted if each successive pair of numbers is in nondecreasing order. For example, if a variable called `list` stores `[3, 8, 2, 5, 19, 24, -3, 0, 4, 4, 8, 205, 42]`, then the call of `list.isPairwiseSorted()` should return `true` because the successive pairs of this list are all sorted: `(3, 8)`, `(2, 5)`, `(19, 24)`, `(-3, 0)`, `(4, 4)`, `(8, 205)`. The extra value 42 at the end had no effect on the result because it is not part of a pair. If the list had instead stored `[7, 42, 308, 409, 19, 17, 2]`, then the method should return `false` because the pair `(19, 17)` is not in sorted order. If a list is so short that it has no pairs, then it is considered to be pairwise sorted.
8. Write a method called `count` that accepts an element value as a parameter and returns the number of occurrences of that value in the list. For example, suppose a variable named `list` stores `[2, -3, 2, 0, 5, 2, 2, 6]`. A call of `list.count(2)` should return 4 because there are four occurrences of that value in the list.
9. Write a method called `maxCount` that returns the number of occurrences of the most frequently occurring value in a sorted list of integers. Because the list will be sorted, all duplicates will be grouped together,

which will make it easier to count duplicates. For example, if a variable called `list` stores `[1, 3, 4, 7, 7, 7, 7, 9, 9, 11, 13, 14, 14, 14, 16, 16, 18, 19, 19, 19]`, the call of `list.maxCount()` should return 4 because the most frequently occurring value (7) occurs four times. It is possible that there will be a tie for the most frequently occurring value, but that doesn't affect the outcome because you are just returning the count, not the value. If there are no duplicates in the list, then every value will occur exactly once and the max count is 1. If the list is empty, the method should return 0.

10. Write a method called `longestSortedSequence` that returns the length of the longest sorted sequence within a list of integers. For example, if a variable called `list` stores `[1, 3, 5, 2, 9, 7, 23, 0, 42, 308, 17]`, then the call of `list.longestSortedSequence()` would return 4 because it is the length of the longest sorted sequence within this list (the sequence `-3, 0, 42, 308`). If the list is empty, your method should return 0. Notice that for a nonempty list the method will always return a value of at least 1 because any individual element constitutes a sorted sequence.
11. Write a method called `removeLast` that removes and returns the last value from a list of integers. For example, if a variable called `list` stores `[8, 17, 42, 3, 8]`, a call of `list.removeLast();` should return 8 and change the list's state to `[8, 17, 42, 3]`. The next call would return 3 and remove 3 from the list, and so on. If the list is empty, throw a `NoSuchElementException`.
12. Write a method called `removeFront` that takes an integer n as a parameter and that removes the first n values from a list of integers. For example, if a variable called `list` stores `[8, 17, 9, 24, 42, 3, 8]` and a call of `list.removeFront(4);` is made, the list's contents should become `[42, 3, 8]`. You may assume that the parameter value passed is between 0 and the size of the list inclusive.
13. Write a method `removeAll` that accepts an integer value as a parameter and that removes all occurrences of the given value from the list.
14. Write a method called `printInversions` that lists all inversions in a list

of integers. An inversion is a pair of numbers in which the first appears before the second in the list, but the first is greater than the second.

Thus, for a sorted list such as [1, 2, 3, 4] there are no inversions at all, and the method would produce no output. Suppose that a variable called `list` stores the values [4, 3, 2, 1]. The call of `list.printInversions()`; would print many inversions:

```
(4, 3)
(4, 2)
(4, 1)
(3, 2)
(3, 1)
(2, 1)
```

The inversions can appear in any order, so this is just one possible correct output. You must reproduce this format exactly, but the inversions can appear in any order. You may assume that the list has no duplicates.

15. Write a method called `mirror` that doubles the size of a list by appending the mirror image of the original sequence to the end of the list. The mirror image is the same sequence of values in reverse order. For example, if a variable called `list` stores [1, 3, 2, 7] and the client calls `list.mirror()`; then the list should be changed to store [1, 3, 2, 7, 7, 2, 3, 1]. Notice that it has doubled in size because the original sequence now appears in reverse order at the end of the list.
16. Write a method called `stutter` that replaces every value with two of that value. For example, if the list initially stores [42, 7, 0, -3, 15], after the call it should store [42, 42, 7, 7, 0, 0, -3, -3, 15, 15].
17. Write a method called `stretch` that takes an integer n as a parameter and that increases a list of integers by a factor of n by replacing each integer in the original list with n copies of that integer. For example, if a variable called `list` stores [18, 7, 4, 24, 11] and we make the call of `list.stretch(3)`; the list should be changed to store [18, 18, 18, 7, 7, 7, 4, 4, 4, 24, 24, 24, 11, 11, 11]. If n is zero or negative, the list should become empty.

18. Write a method called `doubleList` that doubles the size of a list by appending a copy of the original sequence to the end of the list. For example, if the list stores `[1, 8, 2, 7]`, your method should change it to store `[1, 8, 2, 7, 1, 8, 2, 7]`.
19. Write a method called `compress` that replaces every pair of elements in the list with a single element equal to the sum of the pair. If the list is of odd size, leave the last element unchanged. For example, if the list stores `[1, 7, 3, 9, 4, 6, 5]`, your method should change it to store `[8, 12, 10, 5]` (117, then 319, then 416, then 5).
20. Write a method called `rotate` that moves the value at the front of a list of integers to the end of the list. For example, if a variable called `list` stores `[8, 23, 19, 7, 12, 4]`, the call of `list.rotate();` should move the value 8 from the front of the list to the back of the list, producing `[23, 19, 7, 12, 4, 8]`.
21. Write a method called `switchPairs` that switches the order of values in the list in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, switch the order of the next two, and so on. If the list contains an odd number of values, the final element is not moved. For example, if the list initially stores `[10, 25, 31, 47, 52, 68, 77]`, your method should switch the first pair (10 and 25), the second pair (31 and 47), and the third pair (52 and 68) to yield `[25, 10, 47, 31, 68, 52, 77]`.

Exercises

Each of the following exercises is a method to be added to the `LinkedList` class from this chapter.

1. Write a method called `set` that accepts an index and a value and sets the list's element at that index to have the given value. You may assume that the index is between 0 (inclusive) and the size of the list (exclusive).
2. Write a method called `min` that returns the minimum value in a list of integers. If the list is empty, it should throw a `NoSuchElementException`.
3. Write a method called `isSorted` that returns `true` if the list is in sorted (nondecreasing) order and returns `false` otherwise. An empty list is considered to be sorted.
4. Write a method called `lastIndexOf` that accepts an integer value as a parameter and that returns the index in the list of the last occurrence of that value, or -1 if the value is not found in the list. For example, if a variable `list` stores the values `[1, 18, 2, 7, 18, 39, 18, 40]`, then the call of `list.lastIndexOf(18)` should return 6. If the call had instead been `list.lastIndexOf(3)`, the method would return -1.
5. Write a method called `countDuplicates` that returns the number of duplicates in a sorted list. The list will be in sorted order, so all of the duplicates will be grouped together. For example, if a variable `list` stores the values `[1, 1, 1, 3, 3, 6, 9, 15, 15, 23, 23, 23, 40, 40]`, the call of `list.countDuplicates()` should return 7 because there are 2 duplicates of 1, 1 duplicate of 3, 1 duplicate of 15, 2 duplicates of 23, and 1 duplicate of 40.
6. Write a method called `hasTwoConsecutive` that returns whether or not a list of integers has two adjacent numbers that are consecutive integers (`true` if such a pair exists and `false` otherwise). For example, if a

variable `list` stores the values `[1, 18, 2, 7, 8, 39, 18, 40]`, then the call `list.hasTwoConsecutive()` should return `true` because the list contains the adjacent numbers `(7, 8)`, which are a pair of consecutive numbers.

7. Write a method called `deleteBack` that deletes the last value (the value at the back of the list) and returns the deleted value. If the list is empty, throw a `NoSuchElementException`.
8. Write a method called `switchPairs` that switches the order of values in the list in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, switch the order of the next two, and so on. If the list contains an odd number of values, the final element is not moved. For example, if the list initially stores `[10, 25, 31, 47, 52, 68, 77]`, your method should switch the first pair (10 and 25), the second pair (31 and 47), and the third pair (52 and 68) to yield `[25, 10, 47, 31, 68, 52, 77]`.
9. Write a method called `stutter` that doubles the size of a list by replacing every integer in the list with two of that integer. For example, suppose a variable `list` stores the values `[1, 8, 19, 4, 17]`, after a call of `list.stutter()`, it should store `[1, 1, 8, 8, 19, 19, 4, 4, 17, 17]`.
10. Write a method called `stretch` that takes an integer n as a parameter and that increases a list of integers by a factor of n by replacing each integer in the original list with n copies of that integer. For example, if a variable called `list` stores `[18, 7, 4, 24, 11]` and we make the call of `list.stretch(3)`; the list should be changed to store `[18, 18, 18, 7, 7, 7, 4, 4, 4, 24, 24, 24, 11, 11, 11]`. If n is zero or negative, the list should become empty.
11. Write a method called `compress` that replaces every pair of elements in the list with a single element equal to the sum of the pair. If the list is of odd size, leave the last element unchanged. For example, if the list stores `[1, 7, 3, 9, 4, 6, 5]`, your method should change it to store `[8, 12, 10, 5]` (117, then 319, then 416, then 5).

12. Write a method called `split` that rearranges the elements of a list so that all of the negative values appear before all of the nonnegatives. For example, suppose a variable `list` stores the values `[8, 7, -4, 19, 0, 43, -8, -7, 2]`. The call of `list.split();` should rearrange the list to put the negatives first: `[-4, -8, -7, 8, 7, 19, 0, 43, 2]`. It doesn't matter what order the numbers are in, only that the negatives appear before the nonnegatives, so this is only one possible solution. You must solve the problem by rearranging the links of the list, not by swapping data values or creating new nodes. You also may not use auxiliary structures like arrays or `ArrayLists` to solve this problem.
13. Write a method called `transferFrom` that accepts a second linked list as a parameter and that moves values from the second list to this list. You are to attach the second list's elements to the end of this list. You are also to empty the second list. For example, suppose two lists called `list1` and `list2` store `[8, 17, 2, 4]` and `[1, 2, 3]`, respectively. The call of `list1.transferFrom(list2);` should change `list1` to `[8, 17, 2, 4, 1, 2, 3]` and `list2` to an empty list, `[]`. The order of the arguments matters; `list2.transferFrom(list1);` should change `list1` to an empty list, `[]`, and `list2` to `[1, 2, 3, 8, 17, 2, 4]`. Either of the two lists could be empty, but you can assume that neither list is `null`. You are not to create any new nodes. Your method should simply change links of the lists to join them together.
14. Write a method called `removeAll` that removes all occurrences of a particular value. For example, if a variable `list` stores the values `[3, 9, 4, 2, 3, 8, 17, 4, 3, 18]`, the call of `list.removeAll(3);` would change the list to store `[9, 4, 2, 8, 17, 4, 18]`.
15. Write a method called `equals` that accepts a second list as a parameter, returns `true` if the two lists are equal, and returns `false` otherwise. Two lists are considered equal if they store exactly the same values in exactly the same order and have exactly the same length.
16. Write a method called `removeEvens` that removes the values in even-numbered indexes from a list, returning a new list that contains those values in their original order. For example, consider a variable `list1` that stores the values `[8, 13, 17, 4, 9, 12, 98, 41, 7, 23, 0,`

92] and imagine that the following call is made:

```
LinkedList list2 = list1.removeEvens();
```

After the call, `list1` should store [13, 4, 12, 41, 23, 92] and `list2` should store [8, 17, 9, 98, 7, 0]. You may not call any methods of the class other than the constructor to solve this problem. You may not create any new nodes nor change the values stored in data fields to solve this problem. You must solve it by rearranging the links of the list.

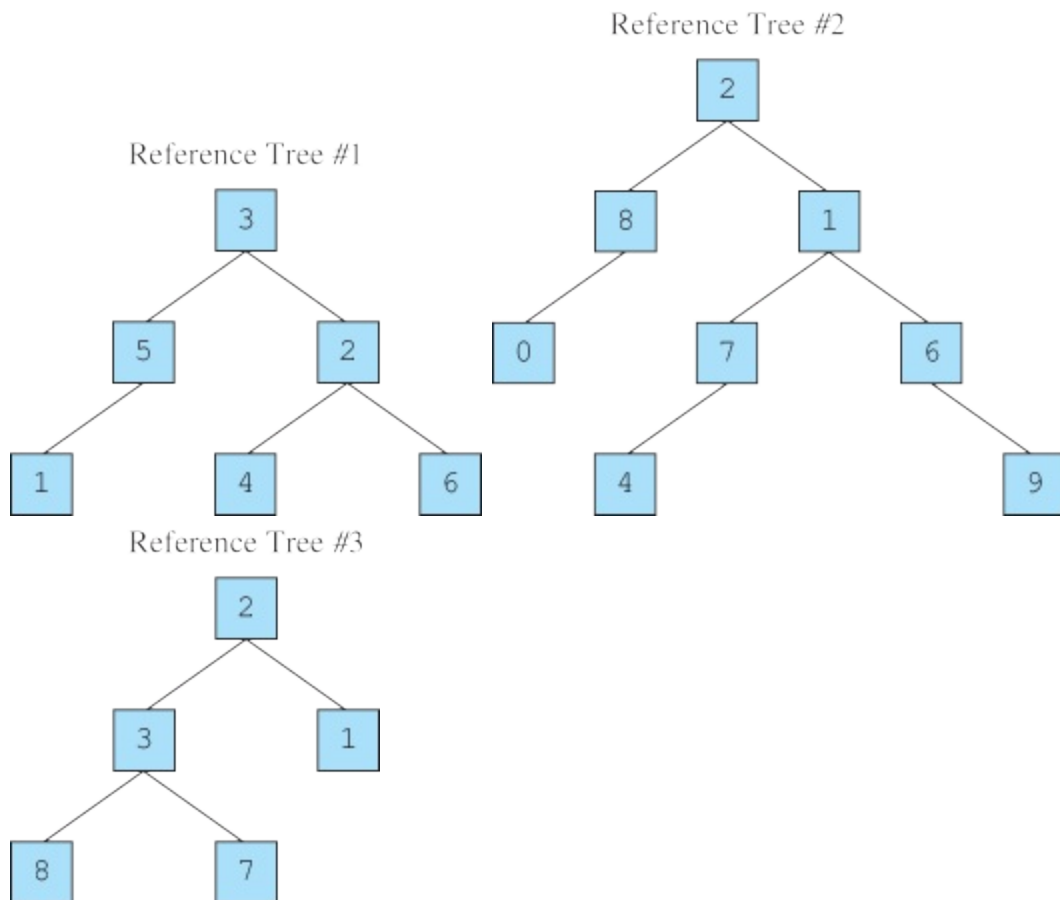
17. Write a method called `removeRange` that accepts a starting and ending index as parameters and removes the elements at those indexes (inclusive) from the list. For example, if a variable `list` stores the values [8, 13, 17, 4, 9, 12, 98, 41, 7, 23, 0, 92], the call of `listRange.removeRange(3, 8);` should remove values between index 3 and index 8 (the values 4 and 7), leaving the list of [8, 13, 17, 23, 0, 92]. The method should throw an `IllegalArgumentException` if either of the positions is negative. Otherwise you may assume that the positions represent a legal range of the list (0 # start # end , size).
18. Write a method called `doubleList` that doubles the size of a list by appending a copy of the original sequence to the end of the list. For example, if a variable `list` stores the values [1, 3, 2, 7] and we make the call of `list.doubleList();` then after the call it should store [1, 3, 2, 7, 1, 3, 2, 7]. Notice that the list has been doubled in size by having the original sequence appear twice in a row. You may not make assumptions about how many elements are in the list. You may not call any methods of the class to solve this problem. If the original list contains N nodes, then you should construct exactly N nodes to be added. You may not use any auxiliary data structures such as arrays or `ArrayLists` to solve this problem. Your method should run in $O(N)$ time where N is the number of nodes in the list.
19. Write a method called `rotate` that moves the value at the front of a list of integers to the end of the list. For example, if a variable called `list` stores the values [8, 23, 19, 7, 45, 98, 102, 4], then the call of `list.rotate();` should move the value 8 from the front of the list to the back of the list, changing the list to store [23, 19, 7, 45, 98, 102,

4, 8]. If the method is called for a list of 0 elements or 1 element, it should have no effect on the list. You may neither construct any new nodes to solve this problem nor change any of the data values stored in the nodes. You must solve the problem by rearranging the links of the list.

20. Write a method called `shift` that rearranges the elements of a list of integers by moving to the end of the list all values that are in odd-numbered positions and otherwise preserving list order. For example, suppose that a variable `list` stores the values [10, 31, 42, 23, 44, 75, 86]. The call of `list.shift()`; should rearrange the list to store [10, 42, 44, 86, 31, 23, 75]. It doesn't matter whether the value itself is odd or even; what matters is whether the value appears in an odd index (index 1, 3, 5, etc.). Also notice that the original order of the list is otherwise preserved. You may not construct any new nodes nor use any auxiliary data structures to solve this problem. You also may not change any data fields of the nodes; you must solve this problem by rearranging the links of the list.
21. Write a method called `reverse` that reverses the order of the elements in the list. (This is very challenging!) For example, if the variable `list` initially stores the values [1, 8, 19, 4, 17], the call of `list.reverse()`; should change the list to store [17, 4, 19, 8, 1].

Exercises

Each of the following exercises is a method to be added to the `IntTree` class from this chapter. You may define additional private methods to implement your public method if necessary. Several problem descriptions refer to the following reference binary trees:



1. Write a method called `countLeftNodes` that returns the number of left children in the tree. A left child is a node that appears as the root of the left-hand subtree of another node. For example, reference tree #1 has 3 left children (the nodes storing the values 5, 1, and 4).
2. Write a method called `countEmpty` that returns the number of empty branches in a tree. An empty tree is considered to have one empty branch (the tree itself). For nonempty trees, your methods should count

the total number of empty branches among the nodes of the tree. A leaf node has two empty branches, a node with one nonempty child has one empty branch, and a node with two nonempty children has no empty branches. For example, reference tree #1 has 7 empty branches (two under the value 1, one under 5, and two under each of 4 and 6).

3. Write a method called `depthSum` that returns the sum of the values stored in a binary tree of integers weighted by the depth of each value. The method should return the value at the root, plus 2 times the values stored at the next level of the tree, plus 3 times the values stored at the next level of the tree, and so on. For example, the depth sum of reference tree #1 would be computed as $(1 * 3) + (2 * (5 + 2)) + (3 * (1 + 4 + 6)) = 50$.
4. Write a method called `countEvenBranches` that returns the number of branch nodes in a binary tree that contain even numbers. A branch node has one or two children (i.e., it is not a leaf). For example, if a variable `t` refers to reference tree #2, then the call `t.countEvenBranches()` should return 3 because there are three branch nodes with even values (2, 8, and 6). Notice that leaf nodes with even values are not included (the nodes storing 0 and 4).
5. Write a method called `printLevel` that accepts an integer parameter `n` and prints the values at level `n` from left to right, one per line. We will use the convention that the overall root is at level 1, its children are at level 2, and so on. If there are no values at the level, your method should produce no output. Your method should throw an `IllegalArgumentException` if it is passed a value for a level that is less than 1. For example, if a variable `t` refers to reference tree #2, then the call of `t.printLevel(3);` would produce the following output:

0
7
6
6. Write a method called `printLeaves` that prints to `System.out` the leaves of a binary tree from right to left. More specifically, the leaves should be printed in the *reverse* order that they would be printed using any of the

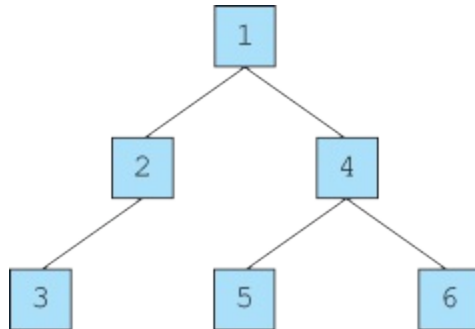
standard traversals. If the tree is empty, your method should produce the output "no leaves". For example, if a variable `t` refers to reference tree #2, the call of `t.printLeaves()`; should produce the following output:

```
leaves: 9 4 0
```

7. Write a method called `isFull` that returns `true` if a binary tree is full and `false` if it is not. A full binary tree is one in which every node has 0 or 2 children. For example, reference trees #1 and #2 are not full, but #3 is full. By definition, the empty tree is considered full.
8. Write a `toString` method for a binary tree of integers. The method should return "empty" for an empty tree. For a leaf node, it should return the data in the node as a string. For a branch node, it should return a parenthesized `String` that has three elements separated by commas: the data at the root, a string representation of the left subtree, and then a string representation of the right subtree. For example, if a variable `t` refers to reference tree #2, then the call `t.toString()` should return the following `String` (without the surrounding quotes):

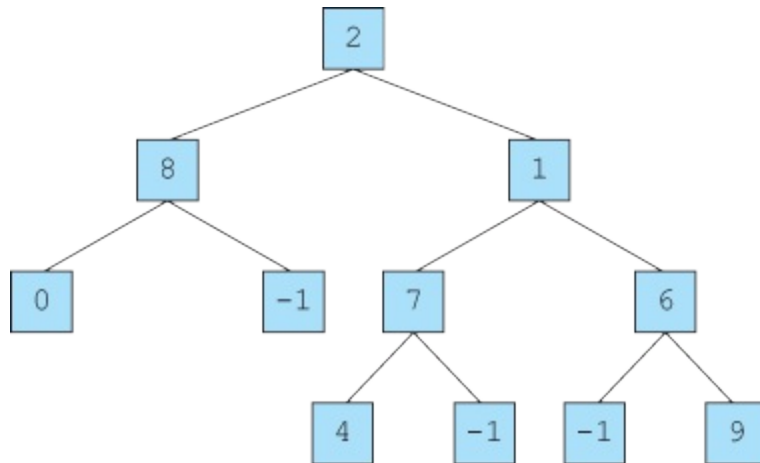
```
"(2, (8, 0, empty), (1, (7, 4, empty), (6, empty, 9)))"
```
9. Write a method called `equals` that accepts another binary tree of integers as a parameter and compares the two trees to see whether they are equal to each other. For example, if variables of type `IntTree` called `t1` and `t2` have been initialized, then `t1.equals(t2)` will return `true` if the trees are equal and `false` otherwise. Two empty trees are considered to be equal to each other.
10. Write a method called `doublePositives` that doubles all data values greater than 0 in a binary tree of integers.
11. Write a method called `numberNodes` that changes the data stored in a binary tree, assigning sequential integers starting with 1 to each node so that a preorder traversal will produce the numbers in order (1, 2, 3, etc.). For example, if a variable `t` refers to reference tree #1, the call of `t.numberNodes()`; would overwrite the existing data, assigning values from 1 to 6 to the nodes so that a preorder traversal of the tree would

produce 1, 2, 3, 4, 5, 6 as shown in the following diagram. Do not change the structure of the tree, only the values stored in the data fields. Your method should return the number of nodes in the tree.

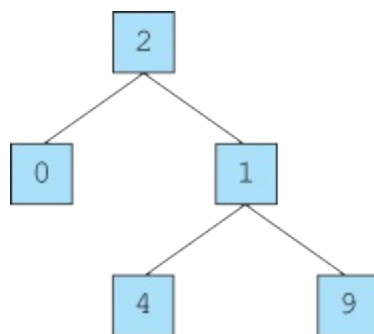


12. Write a method called `removeLeaves` that removes the leaves from a tree. A leaf is a node that has empty left and right subtrees. If your method is called on an empty tree, the method does not change the tree because there are no nodes of any kind (leaf or not).
13. Write a method called `copy` that returns a reference to a new `IntTree` that is an independent copy of the original tree. Do not change the original tree.
14. Write a method called `completeToLevel` that accepts an integer n as a parameter and that adds nodes to a tree to complete the first n levels. A level is complete if every possible node at that level is not `null`. We will use the convention that the overall root is at level 1, its children are at level 2, and so on. You should preserve any existing nodes in the tree. Any new nodes added to the tree should contain the value `-1`. Your method should throw an `IllegalArgumentException` if it is passed a value for a level that is less than 1.

For example, if a variable called `t` refers to reference tree #2 and you make the call of `t.completeToLevel(4);`, the tree should change to the following tree:



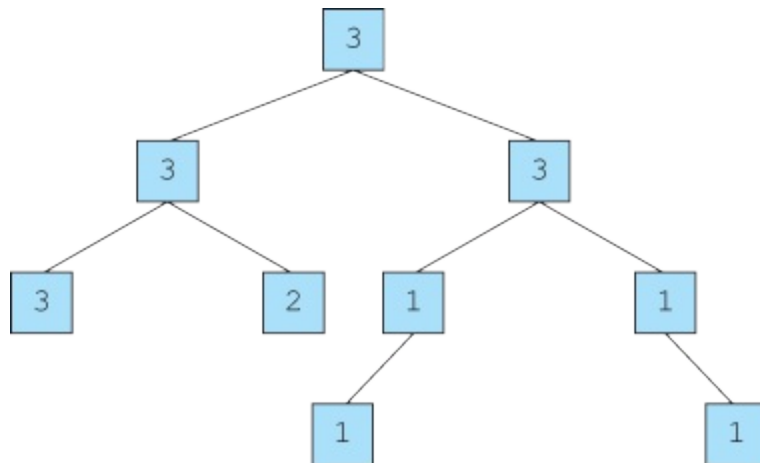
15. Write a method called `trim` that accepts minimum and maximum integers as parameters and removes from the tree any elements that are not within that range inclusive. For this method you should assume that your tree is a binary search tree and that its elements are in valid binary search tree order. Your method should maintain the binary search tree ordering property of the tree. This property is important for solving this problem.
16. Write a method called `tighten` that eliminates branch nodes that have only one child. Each such node should be replaced by its only child. (This can lead to multiple replacements because the child might itself be replaced.) For example, if a variable called `t` refers to reference tree #2, the call of `t.tighten();` should leave `t` storing the following tree.



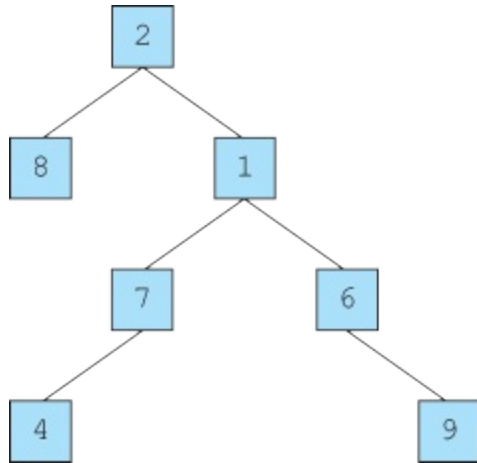
17. Write a method called `combinewith` that accepts another binary tree of integers as a parameter and combines the two trees into a new third tree that is returned. The new tree's structure should be a union of the structures of the two original trees; it should have a node in any location where there was a node in either of the original trees (or both). The

nodes of the new tree should store an integer indicating which of the original trees had a node at that position (1 if just the first tree had the node, 2 if just the second tree had the node, and 3 if both trees had the node). Your method should not change the structure or contents of either of the two original trees that are being combined.

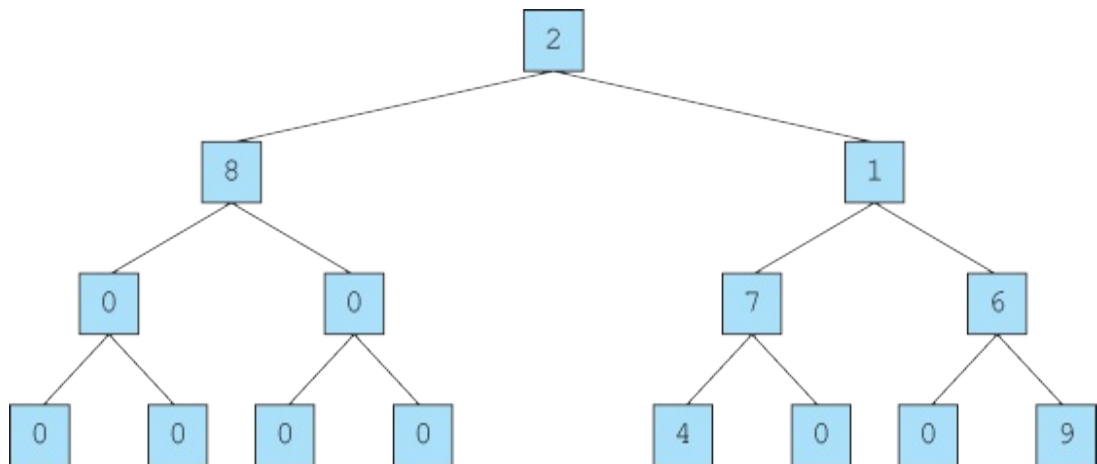
For example, suppose `IntTree` variables `t2` and `t3` refer to reference trees #2 and #3, respectively. The call of `t2.combineWith(t3)` will return a reference to the following new tree. Keep in mind that nodes numbered 1 are those that appear only in `t2`, nodes numbered 2 appear only in `t3`, and nodes numbered 3 appear in both.



18. Write a method called `inOrderList` that returns a list containing the sequence of values obtained from an inorder traversal of your binary tree of integers. For example, if a variable `t` refers to reference tree #3, then the call `t.inOrderList()` should return the list `[8, 3, 7, 2, 1]`. If the tree is empty, your method should return an empty list.
19. Write a method called `evenLevels` that makes sure that all branches end on an even level. If a leaf node is on an odd level it should be removed from the tree. We will define the root as being on level 1. For example, if a variable `t` refers to reference tree #2, then the call `t.evenLevels()`; should change the tree's state to the following, removing the leaf value 0 because it is on an odd level:



20. Write a method called `makePerfect` that adds nodes until the binary tree is a *perfect* tree. A perfect binary tree is one where all leaves are at the same level. Another way of thinking of it is that you are adding dummy nodes to the tree until every path from the root to a leaf is the same length. A perfect tree's shape is exactly triangular and every branch node has exactly two children, and all of the leaves are at the same level. Each new node you add to the tree should store the value 0. For example, if a variable `t` refers to reference tree #2, then the call `t.makePerfect()`; should change the tree's state to the following:



Exercises

1. Write a method in the `HashSet` class called `addAll` that accepts another hash set as a parameter and adds all of the elements from the other set into the current set. For example, if the set stores `[25, 1, 2, 3]` and the method is passed `[2, 3, 6, 44, 79]`, your set would store `[25, 1, 2, 3, 6, 44, 79]`.
2. Write a method in the `HashSet` class called `containsAll` that accepts another hash set as a parameter and returns `true` if your set contains every element from the other set. For example, if the set stores `[22, 3, 5, 6, 8]` and the method is passed `[3, 6, 8]`, your method would return `true`. If the method were passed `[3, 6, 7, 8]`, your method would return `false` because your set does not contain the value 7.
3. Write a method in the `HashSet` class called `equals` that accepts another hash set as a parameter and returns `true` if the two sets contain exactly the same elements. The internal hash table size and ordering of the elements does not matter, only the sets of elements themselves.
4. Write a method in the `HashSet` class called `removeAll` that accepts another hash set as a parameter and ensures that this set does not contain any of the elements from the other set. For example, if the set stores `[22, 3, 5, 6, 8]` and the method is passed `[2, 3, 6, 8, 11]`, your set would store `[22, 5]`.
5. Write a method in the `HashSet` class called `retainAll` that accepts another hash set as a parameter and removes all elements from this set that are not contained in the other set. For example, if the set stores `[22, 3, 5, 6, 8]` and the method is passed `[2, 3, 6, 8, 11]`, your set would store `[3, 6, 8]`.
6. Write a method in the `HashSet` class called `toArray` that returns the elements of the set as a filled array. The order of the elements in the array is not important as long as all elements from the set are present in

the array, with no extra empty slots before or afterward.

7. Write a method in the `HashSet` class called `toString` that returns a string representation of the elements in the set, such as "[22, 3, 5, 6, 8]". The order of the elements in the string does not matter as long as they are all present in the proper format. Do not list any empty or meaningless indexes in the string.
8. Write a method called `descending` that accepts an array of integers and rearranges the integers in the array to be in descending order using a `PriorityQueue` as a helper. For example, if the array passed stores [42, 9, 22, 17, 23, 81], after the call the array should store [81, 42, 22, 17, 9, 23].
9. Write a method called `kthSmallest` that accepts a `PriorityQueue` of integers and an integer *k* as parameters and returns the *k*th-smallest integer from the priority queue. For example, if the queue passed stores the integers [42, 50, 45, 78, 61] and *k* is 4, return the fourth-smallest integer, which is 61. If *k* is 0 or negative or greater than the size of the queue, throw an `IllegalArgumentException`. If your method modifies the state of the queue during its computation, it should restore the queue before it returns. You may use one stack or queue as auxiliary storage.
10. Write a method called `isConsecutive` that accepts a `PriorityQueue` of integers as a parameter and returns `true` if the queue contains a sequence of consecutive integers starting from the front of the queue. Consecutive integers are integers that come one after the other, as in 5, 6, 7, 8, 9, etc., so if the queue stores [7, 8, 9, 10, 11], your method should return `true`. If your method modifies the state of the queue during its computation, it should restore the queue before it returns. You may use one stack or queue as auxiliary storage.
11. Write a method called `removeDuplicates` that accepts a `PriorityQueue` of integers as a parameter and modifies the queue's state so that any element that is equal to another element in the queue is removed. For example, if the queue stores [7, 7, 8, 8, 8, 10, 45, 45], your method should modify the queue to store [7, 8, 10, 45]. You may use one stack or queue as auxiliary storage.

12. Write a method called `stutter` that accepts a `PriorityQueue` of integers as a parameter and replaces every value in the queue with two occurrences of that value. For example, if the queue stores `[7, 8, 10, 45]`, your method should modify the queue to store `[7, 7, 8, 8, 10, 10, 45, 45]`. You may use one stack or queue as auxiliary storage.
13. Write a method in the `HeapIntPriorityQueue` class called `toArray` that returns the elements of the queue as a filled array. The order of the elements in the array is not important as long as all elements from the queue are present in the array, with no extra empty slots before or afterward.
14. Write a method in the `HeapIntPriorityQueue` class called `toString` that returns a string representation of the elements in the queue, such as `"[42, 50, 45, 78, 61]"`. The order of the elements in the string does not matter as long as they are all present in the proper format.
15. Write a method in the `HeapIntPriorityQueue` class called `merge` that accepts another `HeapIntPriorityQueue` as a parameter and adds all elements from the other queue into the current queue, maintaining proper heap order such that the elements will still come out in ascending order when they are removed. Your code should not modify the queue passed in as a parameter. (Recall that objects of the same class can access each other's private fields.)

Programming Projects

1. Add an iterator to the `HashSet` class written in this chapter. To do this you will need to write an inner class that can iterate over the elements of the set, remembering its position as it moves along. Also write a method in the `HashSet` class called `iterator` that returns such an iterator object.
2. In this chapter, we implemented a `HashSet` that used separate chaining to resolve collisions. Implement your own version of `HashSet` that uses linear probing to resolve collisions. Perform lazy removals, placing a special “removed” marker value into any bucket where an element is removed.
3. Modify the `HeapIntPriorityQueue` class written in this chapter to make it configurable in ways similar to Java's `PriorityQueue` class. Make it possible for the heap to be a min-heap or max-heap. (If you create a heap of objects, you could also modify it to accept a `Comparator` parameter to its constructor.)
4. Modify the `HeapIntPriorityQueue` class written in this chapter to make it into a three-heap. A three-heap is similar to a binary heap, except that each node is considered to have three children rather than two children. A three-heap is a wider tree though not as tall as a binary heap. The heap can still be represented as an array, but the algorithms for traversing it must be modified to consider all three children when bubbling as appropriate, etc. (Do you expect that a three-heap will be faster or slower than a binary heap for insertion, and for removal? Why? You can create a test to verify the performance of each operation.)
5. Write a program that implements the “Huffman coding” compression algorithm using priority queues and binary trees. Huffman coding is an algorithm devised by David A. Huffman of MIT in 1952 for compressing text data to make a file occupy a smaller number of bytes. Normally text data is stored in a standard format of 8 bits per character, commonly using an encoding called ASCII that maps every character to

a binary integer value from 0–255. The idea of Huffman coding is to abandon the rigid 8-bits-per-character requirement and use different-length binary encodings for different characters. The advantage of doing this is that if a character occurs frequently in the file, such as the letter “e”, it could be given a shorter encoding (fewer bits), making the file smaller.

The steps involved in Huffman coding a given text source file into a destination compressed file are the following:

- a. Examine the source file's contents and count the number of occurrences of each character (consider using a map).
- b. Place each character and its frequency (count of occurrences) into a priority queue ordered in ascending order by character frequency.
- c. Convert the contents of this priority queue into a binary tree with a particular structure. Create this tree by repeatedly removing the two front elements from the priority queue (the two nodes with the lowest frequencies) and combining them into a new node with these two nodes as its children and the two nodes' combined frequencies as its frequency. Then reinsert this combined node back into the priority queue. Repeat until the priority queue contains just one single node.
- d. Traverse the tree to discover the binary encodings of each character. Each left branch represents a '0' in the character's encoding and each right branch represents a “1”.
- e. Reexamine the source file's contents, and for each character, output the encoded binary version of that character to the destination file to compress it.

Exercises

1. Write a method `printDoubled` that uses stream operations to print twice the value of each element of array of integers. For example, if the array passed is `{2, -1, 4, 16}`, print `4 -2 8 32`.
2. Write a method `sumAbsVals` that uses stream operations to compute the sum of the absolute values of an array of integers. For example, the sum of `{-1, 2, -4, 6, -9}` is 22.
3. Write a method `largestEven` that uses stream operations to find and return the largest even number from an array of integers. For example, if the array is `{5, -1, 12, 10, 2, 8}`, your method should return 12. You may assume that the array contains at least one even integer.
4. Write a method `totalCircleArea` that uses stream operations to compute and return the sum of the areas of a group of circles, rounded to the nearest whole number. Your function accepts an array of real numbers representing the radii of the circles. For example, if the array is `{3.0, 1.0, 7.2, 5.5}`, return `289.0`. Recall that the area of a circle of radius r is πr^2 .
5. Write a method `countNegatives` that uses stream operations to count how many numbers in a given array of integers are negative. For example, if the array is `{5, -1, -3, 20, 47, -10, -8, -4, 0, -6, -6}`, return 7.
6. Write a method `pigLatin` that uses stream operations to convert a `String` parameter into its “Pig Latin” form. For this problem we'll use a simple definition of Pig Latin where the first letter should be moved to the end of the word and followed by “ay.” For example, if the string passed is `"go seattle mariners"`, return `"o-gay eattle-say ariners-may"`.
7. Write a method `countVowels` that uses stream operations to count the

number of vowels in a given string. A vowel is an A, E, I, O, or U, case-insensitive. For example, if the string is "Soo beautiful", there are seven vowels.

8. Write a method `toSortedForm` that uses stream operations to convert a `String` parameter into a sorted form with its letters in alphabetical order. For example, if the string passed is "tennessee", return "eeeeennsst".
9. Write a method `stdev` that computes the standard deviation of an array of real numbers. The formula for computing a standard deviation s of N values is the following, where x_i represents each i^{th} element and μ represents the arithmetic mean (average) of all the elements:

$$\sigma = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

10. Write a method `glueReverse` that accepts a `List` of strings as its parameter and uses stream operations to return a single string consisting of the list's elements concatenated together in reverse order. For example, if the list stores ["the", "quick", "brown", "fox"], you should return "foxbrownquickthe".
11. Write a method `theLines` that accepts a file name as a parameter and uses stream operations to return a count of the number of lines in the file that start with the word "The", case-insensitive.
12. Write a method `fourLetterWords` that accepts a file name as a parameter and returns a count of the number of unique lines in the file that are exactly four letters long. Assume that each line in the file contains a single word.